Optimizing post-quantum TLS on embedded clients

by

Ganyu Xu

A thesis
presented to the University of Waterloo
in fulfillment of the
thesis requirement for the degree of
Master of Applied Science
in
Electrical and Computer Engineering

Waterloo, Ontario, Canada, 2025

Examining Committee Membership

The following served on the Examining Committee for this thesis. The decision of the Examining Committee is by majority vote.

Supervisor(s):	Guang Gong Professor, Electrical & Computer Engineering University of Waterloo
Internal Member:	Hiren Patel Professor, Electrical & Computer Engineering University of Waterloo
	Jesse Van Griensven Adjunct Professor, Mechanical & Mechatronics Engineering University of Waterloo

Author's Declaration

I hereby declare that I am the sole author of this thesis. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners.

I understand that my thesis may be made electronically available to the public.

Abstract

Transport Layer Security (TLS) is the most widely used cryptographic protocol on the Internet. It ensures the confidentiality, integrity, and authenticity of application data using a combination of cryptographic primitives including Diffie-Hellman key exchange, digital signatures, cryptographic hash functions, and authenticated encryption with associated data (AEAD). Unfortunately, Peter Shor's quantum integer factorization algorithm and recent progress in engineering large-scale quantum computers posed an existential threat to number-theoretic and elliptic-curve public-key cryptographic algorithms used in TLS. The risk of "harvest-now-decrypt-later" attacks and the enormity of efforts needed to migrate existing digital infrastructure meant that we needed to start transitioning to using post-quantum cryptography (PQC) as soon as possible.

Since the earliest public experimentation of adopting PQC in TLS by Google, and the start of NIST's PQC standardization project, a collaboration between government, industry, and academia over the last two decades have produced impressive progress towards a quantum-safe future. As of July 2025, NIST has standardized three PQC algorithms (ML-KEM, ML-DSA, SLH-DSA), and research projects such as Open Quantum Safe (OQS) have integrated PQC algorithms into popular cryptographic protocols (TLS, SSH, VPN, etc.) for experimental deployment and evaluation. While the migration to PQC is gathering momentum, deploying PQC to embedded systems received comparatively less attention despite the proliferation of IoT devices and the growing importance of IoT security. There are fewer readily available embedded TLS libraries with PQC support, and less systematic efforts toward understanding the performance and security impact of deploying post-quantum TLS on embedded clients.

In this work, we made several contributions to understanding and optimizing post-quantum TLS on embedded systems. First, we reduced client's computational workload in ephemeral key exchange by replacing IND-CCA KEM with IND-1CCA KEM. Specifically, we proposed methods for constructing IND-1CCA KEM that avoided the expensive re-encryption technique used in the Fujisaki-Okamoto transformation. Second, we implemented KEM-based authentication (KEMTLS) as an alternative to signature-based authentication in TLS. Compared to signature-based authentication, KEM-based authentication reduces bandwidth requirements and allows the client to start sending application data at an earlier time. Last but not least, we provided a clean, simple implementation of post-quantum TLS and KEMTLS with which we benchmarked the handshake performance on an embedded client. By combining these optimization techniques, we reduced client's TLS handshake latency to 84.17 ms, a 34.4% reduction compared to using elliptic-curve algorithms (128.40 ms).

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

Transport Layer Security (TLS) is the most widely used cryptographic protocol on the Internet. It allows two communicating parties to establish a secure channel over an insecure network. The secure channel then ensures confidentiality, integrity, and authenticity of the application data. *Secure Socket Layer* (SSL), the predecessor of TLS, was originally designed by Netscape (1994) for web browsing (HTTPS), but through more than thirty years of iterations, TLS/SSL has become the default choice for many secure applications, including email (SMTPS, IMAPS, and POP3S), remote desktop (RDP), and sensor networks (MQTT/MQTT-SN).

To achieve its security goals, TLS deploys many cryptographic algorithms: Diffie-Hellman key exchange for establishing a shared secret, digital signatures for entity authentication, cryptographic hash functions for message integrity and key derivation, authenticated encryption with associated data (AEAD) for bulk encryption, etc. Unfortunately, almost all of these cryptographic algorithms are weakened by the discovery of quantum algorithms. For example, Lov Grover proposed a quantum algorithm that could search through a key space of size N using only $N^{1/2}$ steps [42]. Grover's quantum search algorithm can be implemented to weaken the security of symmetric encryption schemes: while a classic computer requires on average 2^{128} steps to brute-force a 128-bit secret key, a quantum computer executing Grover's quantum search algorithm can brute-force it in only 2^{64} steps on average. Grover's search algorithm was later extended by Brassard et al. [11] to reduce the collision resistance of an N-bit hash function from $2^{N/2}$ to $2^{N/3}$. Fortunately, Grover's search algorithm only provided a quadratic speedup, and the security of symmetric primitives can be scaled up straightforwardly. On the other hand, Peter Shor's quantum algorithm for integer factorization [93, 94] provided an *exponential* speedup that would break today's most popular public-key cryptographic algorithms based on number theory or elliptic curves.

The threat of large-scale quantum computer breaking today's encryption demands a swift and complete adoption of post-quantum cryptography (PQC). While quantum computers large enough to solve 2048-bit RSA remains just out of reach with today's technology, the need for PQC is still urgent for two reasons: first, developing new cryptographic primitives and protocols requires extensive research efforts and thorough testing before they are considered ready for production deployment, and rolling out new cryptographic algorithms throughout existing infrastructure takes enormous amount of efforts and time; second, encrypted communication can be intercepted today and its content decrypted sometime later when large quantum computers are feasible, the so-called "harvest-now-decrypt-later" attack.

The growing urgency of transitioning to PQC coincides with the rapidly accelerating deployment of Internet-connected embedded devices (Internet of Things, or IoT), which raised an interesting but often overlooked problem of post-quantum IoT security. Deploying PQC in IoT devices carries many challenges not found in the discussion involving servers, desktops, or even smartphones [67]. IoT devices are comparatively underpowered, with 32-bit or 16-bit processors that have one or two cores clocked at 100-200MHz and a few hundred kilobytes of memory (by comparison, iPhone 13 has six 64-bit cores running at up to 3GHz and 4GB of RAM). Many IoT devices are also battery-powered, with a single charge expected to last months if not years of intermittent operations, making computational efficiency critical to their normal functions. Last but not least, IoT devices are at higher risk of physical intrusion and sophisticated side-channel attacks, which require the use of special hardening techniques not found in desktop environments.

In this thesis, we contributed to the discussion of post-quantum IoT security by implementing, benchmarking, and optimizing post-quantum TLS (PQ-TLS) on embedded systems. Our work demonstrated the feasibility of deploying PQC with very high security level in TLS, and our benchmark data help identify possible trade-offs among the vast selection of possible configurations. Our implementation also served as a starting point for future works evaluating PQ-TLS on other platforms that is easier to work with than prior implementations. We hope our contribution help bridge the gap between cryptographic research and practical deployment, and accelerate the broad adoption of PQC in the IoT landscape.

	OT-ML-KEM-512	OT-ML-KEM-768	OT-ML-KEM-1024
Encap (kilocycles/tick)	93.15 (+1.8%)	146.40 (+7.3%)	205.76 (+3.3%)
Decap (kilocycles/tick)	33.73 (-72.2%)	43.31 (-76.8%)	51.37 (-79.1%)

Table 1.1: OT-ML-KEM reduced decapsulation CPU cycles by more than 70% compared to ML-KEM

Implementation	PQC	Certificates	Client	Server	Protocol
[15, 14]	OQS	OpenSSL	mbedTLS	mbedTLS	TLS 1.2
[106]	OQS	OpenSSL	WolfSSL	WolfSSL	TLS 1.3
[40]	OQS	Python	WolfSSL	rustls	TLS 1.3, KEMTLS
this work	PQClean	WolfSSL	WolfSSL	WolfSSL	TLS 1.3, KEMTLS

Table 1.2: Software dependencies for PQ-TLS/KEMTLS implementations

1.1 Contributions

Our contributions are as follows:

- We propose "encrypt-then-MAC", a KEM transformation that converts an OW-CPA PKE into an IND-1CCA secure KEM. We instantiated this transformation with the CPA sub-routines of ML-KEM, which we named one-time ML-KEM (OT-ML-KEM). Compared to ML-KEM, OT-ML-KEM trades small performance penalty in encapsulation for up to 70% CPU cycle reduction in decapsulation (Table 1.1).

- We implemented PQ-TLS and KEMTLS on top of WolfSSL. Out implementation is the first implementation where certificate generation, client (embedded and desktop), and server are all implemented on the same TLS/SSL library, reducing the complexity of software dependencies and facilitating future evaluation of PQ-TLS/KEMTLS on embedded or desktop environments (Table 1.2).

- We benchmarked handshake performance of PQ-TLS and KEMTLS on an embedded client using a wide selection of classical and post-quantum cryptographic primitives. With a combination of using OT-ML-KEM for ephemeral key exchange and KEM-based authentication, we were able to achieve median client's handshake latency of 84.2 ms, a 34.4% reduction compared to elliptic-curve algorithms (128.40 ms).

1.2 Related works

Post-quantum cryptography Public-key cryptosystems based on other problems than number theory or elliptic curve have been studied well before aforementioned quantum algorithms were published. The earliest examples include the NTRU cryptosystem [46] (based on lattice problems) and the McEliece cryptosystem [69] (based on error correcting codes). Since the publication of Shor's algorithm, the increased interest in developing quantum resistant cryptographic algorithms has produced many additional schemes based on diverse classes of problems. Besides proposals for new lattice-based schemes and code-based schemes, there are novel encryption and signature schemes based on isogeny (SIKE [53], CSIDH [16]), multivariate equations (Rainbow [24], UOV [60]), MPC-in-the-head [51], and symmetric primitives (PICNIC [19], SPHINCS+ [8]).

While several governments and international bodies have ongoing efforts to standardize post-quantum algorithms, in this paper we mainly focus on the PQC standardization project run by the National Institute of Standards and Technology (NIST). Started in 2016, the project called for key encapsulation mechanisms (KEM) and digital signatures, and after several rounds of revisions, NIST announced the selection of four algorithms for standardization in August 2024:

- **ML-KEM** [78] (originally named CRYSTALS-Kyber [10]) is a KEM based on module lattice
- **ML-DSA** [77] (originally named CRYSTALS-Dilithium [29]) is a signature scheme based on module lattice
- **SLH-DSA** [79] (originally named SPHINCS+ [8]) is a signature scheme based on cryptographic hash function
- Falcon [33] is a signature scheme based on NTRU lattice. Although Falcon has been selected for standardization, NIST has not yet published a draft for FIPS 206 (tentatively named FN-DSA).

In March 2025, NIST selected HQC [73]–a KEM based on error-correcting code–for standardization. Additionally, Classic McEliece [7] is under consideration for future selection, and the PQC project is currently calling for and reviewing additional signature schemes as of July 2025 [75].

Following the start of NIST's PQC standardization project, practical implementations of PQC algorithms became more widely available, which motivated many projects to collect individual implementations into a central repository, provide thorough audits and testings, then integrate them into real-world protocols (TLS, SSH, WireGuard, Signal, etc.). PQ-Clean [59] is one such project that curates "reference implementations" (written in ANSI C) and optimized implementations (x86_64 or ARM assembly) submitted to the PQC standardization project. PQClean's maintainers ensure each implementation's correctness and security (free from memory corruption, branching on secrets, etc.), then applies quality-of-life improvements such as a consistent set of API's and namespacing each implementation. PQClean now serves as upstream to other projects that benchmark and integrate PQC, including pqm4 [58, 57] and Open Quantum Safe [100].

PQ-TLS implementations Experimentation with post-quantum TLS began as early as 2016, when Google implemented hybrid key exchange (X25519 + NewHope [1], called CECPQ1) for TLS 1.2 into its Chrome web browser [65]. The main goal of CECPQ1 is to demonstrate the feasibility of adopting post-quantum primitives into the existing web infrastructure (the developers were particularly worried about compatibility with TLS middle boxes, which posed a major obstacle when TLS 1.3 was being rolled out to replace TLS 1.2). It was observed that NewHope carried very little computational overhead, and the increase in message size brought only minimal increase in connection latency: median latency increased by 1 ms, the 95th percentile increased by 20 ms, and the 99th percentile increased by 150 ms. Google and CloudFlare followed up in 2018, when they selected two hybrid key exchange schemes (X25519 + NTRU-HRSS [50] and X25519 + SIKE) and implemented them on TLS 1.3 in Chrome [66]. The CECPQ2 experiment showed that over broadband Internet, the computational performance of the PQC scheme factors more significantly into connection latency than message size, as X25519+SIKE key exchange was significantly slower than X25519+NTRU-HRSS and the control group despite SIKE having much smaller message size.

The availability of individual reference implementations from NIST's PQC project enabled several scatter-shot experiments [25, 56, 97, 98, 99, 45, 54, 87, 71, 92, 83, 96, 55, 82] that deployed a limited set of PQC algorithms and focused only on specific aspects (only PQ key exchange, or only PQ authentication) of TLS. Each experiment used different client/server hardware, different network setups, and different sources of PQC implementations, which made it difficult to compare results across experiments. Paquin et al. [80, 81] leveraged `liboqs` and OpenSSL to provide a more systematic approach that controlled for hardware, network, and software.

Burstinghaus-Steinbach et al. published the earliest implementation of PQ-TLS on embedded clients [15, 14]. This implementation integrated Kyber and SPHINCS+ into

TLS 1.2 using mbedTLS[1], and the test platforms include Raspberry Pi 3B, ESP32-PICO, Fieldbus Option Card, and LPCXpresso. Tasopoulos et al. [105, 106] published a follow-up implementation, this time integrating an expanded selection of NIST PQC Round 3 and 4 candidates into TLS 1.3 on top of WolfSSL. The target platform is a NUCLEO-F439ZI (ARM Cortex-M4). Schoffel et al. [89] and later Tasopoulos et al. [102, 103] also evaluated the energy consumption of post-quantum TLS on battery-powered IoT device, proving the viability of PQC on extremely low-powered devices. Last but not least, Gonzalez and Wiggers [40] implemented KEMTLS on WolfSSL and tested the performance of PQ-TLS and KEMTLS on a Silicon Labs STK3701A board (also called "Giant Gecko").

Experimental results from both industry and acdemia seem to have converged on a consensus regarding the performance impact of adopting PQC in TLS: lattice-based KEM (i.e. ML-KEM/Kyber and NTRU) and signatures (i.e. ML-DSA and Falcon) are computationally as efficient as their elliptic-curve counterparts, and can serve as drop-in replacements. The increased message size can be accommodated within existing TLS specification and network infrastructure, and the performance penalty is minimal on broadband network. Where a pure-quantum TLS is considered too risky, hybrid ECC+PQC key exhange and authentication can be deployed with the guarantee that the overall protocol is secure even if one of the components becomes broken [38, 2, 68, 21], and the performance penalty is non-trivial but still acceptable. Finally, PQC algorithms with conservative security assumptions (hash-based and code-based schemes) are harder to accommodate (e.g. Classic McEliece public key cannot fit into TLS's `key_share` extension), and their performance penalty is too much to justify in real-world use cases.

1.3 Paper organization

The rest of this paper is organized accordingly:

- Chapter 2 reviews TLS/KEMTLS handshake protocol and their security properties.
- Chapter 3 proposes an original IND-1CCA transformation, and discusses its application to TLS key exchange
- Chapter 4 describes how we implemented PQ-TLS/KEMTLS on an embedded client
- Chapter 5 presents performance evaluation of PQ-TLS/KEMTLS on embedded clients
- Chapter 6 summarizes the thesis and discusses open problems and future directions

[1] https://github.com/Mbed-TLS/mbedtls/tree/development/library

Chapter 2

An overview of TLS

Transport Layer Security (TLS) is a communication protocol with which two parties can exchange data in the presence of passive or active adversaries while preserving **confidentiality, integrity, and authenticity**. It was first developed at Netscape in 1994 and went through many iterations. The latest revision is TLS 1.3, formally standardized by IETF in RFC 8446 [86] in 2018. TLS 1.3 made significant improvements over TLS 1.2 [23] and prior versions, including deprecating weak symmetric cipher suites, requiring forward secrecy at all times, and restructuring the handshake state machine in order to simplify the handshake workflow. According to Cloudflare [109], TLS 1.3 reached more than 93% adoption on the Internet. Given TLS 1.3's superior design and nearly universal adoption over older versions of TLS, this work will focus on this version unless otherwise specified.

Section 2.1 reviews TLS 1.3 with an emphasis on the handshake protocol and its security properties. Section 2.2 introduces KEMTLS [90], a proposal to use KEM-based authentication in TLS 1.3.

2.1 TLS 1.3 handshake

TLS 1.3 consists mainly of two sub-protocols. First, client and server establish cryptographic parameters and perform unilateral/mutual authentication in the **handshake** protocol. If the handshake protocol succeeds, then the two parties proceed to exchange **application-layer data** using the secret keys and the authenticated encryption with associated data (AEAD) scheme negotiated in the handshake protocol, otherwise the underlying connection is terminated upon handshake failure. In this thesis, we primarily focus on the handshake sub-protocol. Figure 2.1 illustrates a unilaterally TLS 1.3 handshake over TCP.

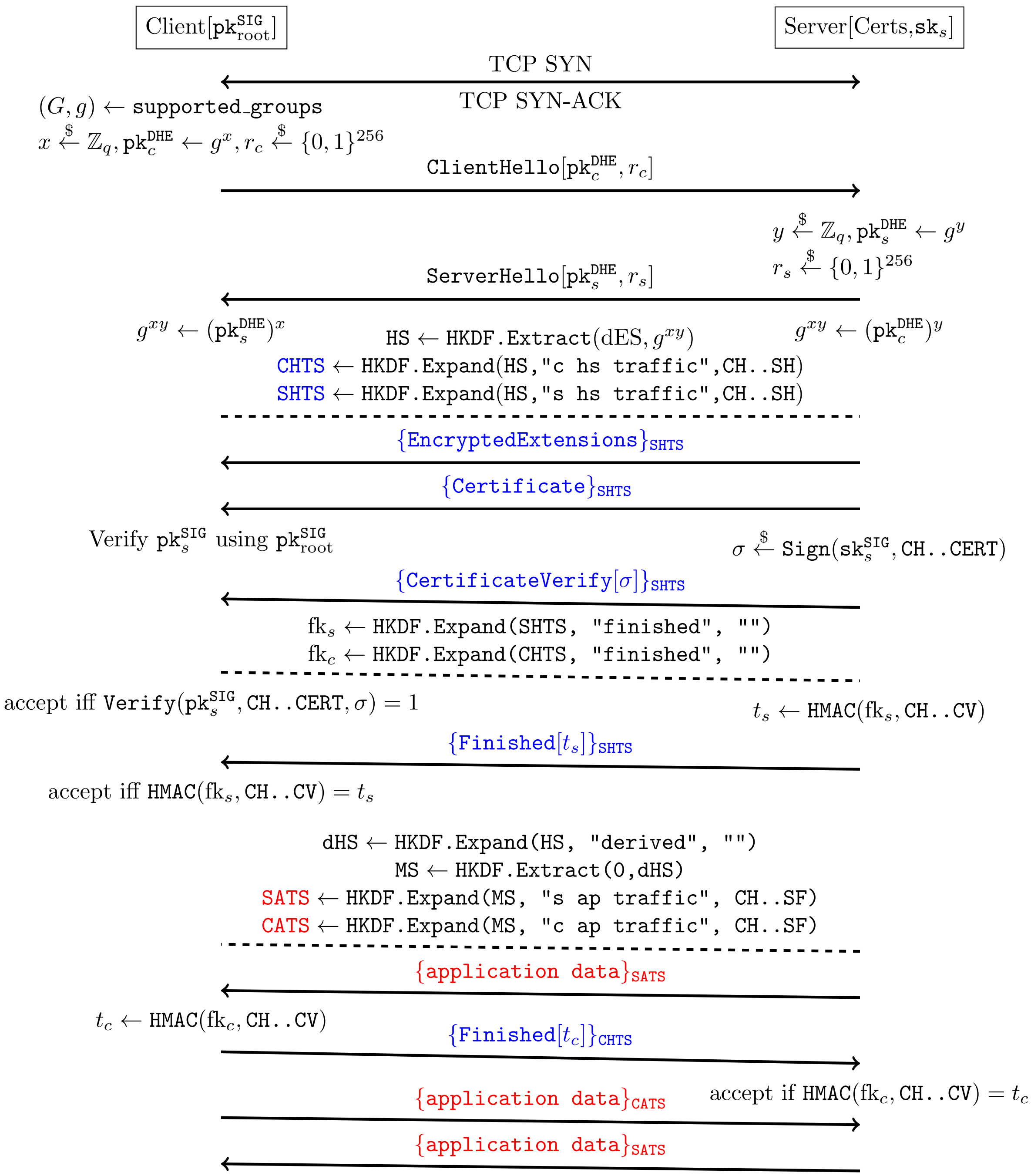

Figure 2.1: TLS 1.3 handshake with unilateral authentication

Ephemeral key exchange To achieve forward secrecy, TLS 1.3 requires using (elliptic-curve) ephemeral Diffie-Hellman key exchange (ECDHE)[1] to establish shared secret between client and server. The ephemeral key exchange is performed with the `ClientHello` and subsequent `ServerHello` messages. First, client chooses one of the pre-defined Diffie-Hellman parameter sets, which typically consists of a group $(G, \cdot)$ and a basepoint $g \in G$ that generates a subgroup of prime order q. The client then samples a random number $x \xleftarrow{\$} \mathbb{Z}_q$ as its ephemeral secret key and computes the corresponding public key $\mathtt{pk}_c^{\mathtt{DHE}}$. Client also samples a 256-bit random number $r_c \xleftarrow{\$} \{0,1\}^{256}$, which is included in `ClientHello` to mitigate replay attacks. Client sends $\mathtt{pk}_c^{\mathtt{DHE}}$ and r_c in `ClientHello` to the server.

If server supports client's choice of (G, g), then it samples another $y \xleftarrow{\$} \mathbb{Z}_q$ as server's secret key, computes $\mathtt{pk}_s^{\mathtt{DHE}} \leftarrow g^y$, and generates its random bits $r_s \xleftarrow{\$} \{0,1\}^{256}$. $\mathtt{pk}_s^{\mathtt{DHE}}$ and r_s are included in `ServerHello` and sent to the client. At this point, server and client can each compute the Diffie-Hellman shared secret: client computes $g^{xy} \leftarrow (\mathtt{pk}_s^{\mathtt{DHE}})^x$ and server computes $g^{xy} \leftarrow (\mathtt{pk}_c^{\mathtt{DHE}})^y$. The shared secret g^{xy} and the nonces r_c, r_s are fed into a key derivation function (KDF) to first derive the *Handshake Secret (HS)*: $\mathtt{HS} \leftarrow \mathtt{HKDF.Extract}(g^{xy}, \mathtt{CH..SH}, \ldots)$. A pair of secret keys called *client handshake traffic secret (CHTS)* and *server handshake traffic secret (SHTS)* are subsequently derived from HS. All remaining handshake messages are encrypted under these two keys: message sent by server is encrypted under SHTS, while message sent by client is encrypted under CHTS. Figure 2.1 color-coded these two secret keys and messages encrypted under them.

Authentication In a unilaterally authenticated TLS handshake, the server needs to prove its identity to the client, which is accomplished with three messages `Certificate`, `CertificateVerify`, and `Finished`. These three messages are sent together, and after sending `Finished`, server can start sending application data. On the other hand, after finishing processing these messages, client will send its own `Finished` and can start sending application data. Hence we say client finishes handshake with one round trip (1-RTT) while server finishes handshake with zero round trip.

The `Certificate` message contains a sequence of X.509 public-key certificates, where each certificate contains some identity, a cryptographic public key, and a digital signature (see Appendix A for an example). The certificate chain binds a cryptographic public key $\mathtt{pk}_s^{\mathtt{SIG}}$ to the server's identity (details are discussed in Section 2.1.1), and the server authenticates itself by proving ownership of the corresponding secret key $\mathtt{pk}_s^{\mathtt{SIG}}$. TLS 1.3 requires signature-based authentication, so server proves ownership by signing the hand-

[1]Finite-field Diffie-Hellman is supported but very rarely used

shake transcript: $\sigma \xleftarrow{\$} \texttt{SIG.Sign}(\texttt{sk}_s^{\texttt{SIG}}, \texttt{CH..CERT})$. σ is included in `CertificateVerify` and sent to the client. Server's final handshake message is `Finished`, which contains an HMAC tag $t_s \leftarrow \texttt{HMAC}(\text{fk}_s, \texttt{CH..CV})$ computed over the handshake transcript. The HMAC secret key fk_s are derived from SHTS. The `Finished` message ensures the integrity of the handshake and of the computed keys. If the client accepts server's `Certificate`, `CertificateVerify`, and `Finished`, then it sends its own `Finished` containing an HMAC tag $t_c \leftarrow \texttt{HMAC}(\text{fk}_c, \texttt{CH..SF})$ computer under a secret key fk_c derived from CHTS.

At the end of the handshake, client and server each updates its internal secrets and derive another pair of secret keys called *client application traffic keys (CATS)* and *server application traffic keys (SATS)*. This process is discussed in details in Section 2.1.2. Application data sent by client and server is respectively encrypted using CATS and SATS.

2.1.1 Public-key Infrastructure (PKI)

A server's cryptographic public key is bound to its identity using X.509 public-key certificates. Figure 2.2 illustrates a certificate chain.

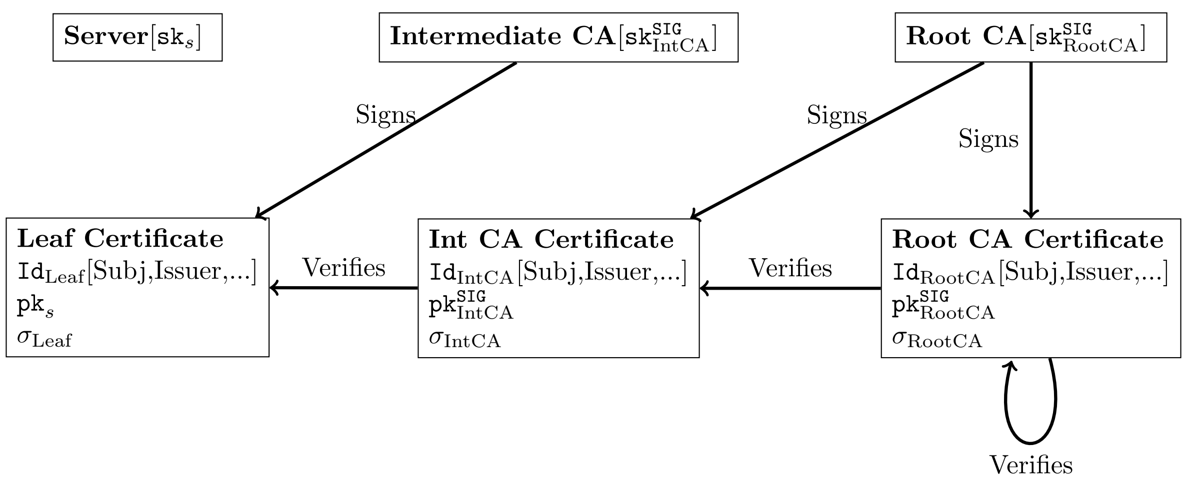

Figure 2.2: Certificate chain and PKI

Each X.509 certificate encodes three pieces of information: identities of the issuer and the subject, a cryptographic public key, and a cryptographic signature over the identity and the public key. Through this signature, the issuer testifies the subject's ownership of the public key and binds the public key to the identity. The signature of the issuer is verified using the issuer's public key, which is usually presented on another public-key certificate. This creates a chain of trust in which each certificate is verified by the public key in another certificate, up to the root of trust, which is usually a self-signed certificate whose signature is verified by the public key on the same certificate.

The *Public Key Infrastructure (PKI)* consists of entities that play different roles in this chain of trust. The server (and optionally client) is the leaf node of the chain, and the entities issuing certificates are called *Certificate Authorities (CA)*. The root of this chain is thus referred to as *root CA*, while other issuers between the leaf and the root are referred to as intermediate CAs. A selection of trusted root CA certificates are typically pre-installed in client devices by the vendors (e.g. Google, Mozilla, Apple, Microsoft, etc.).

2.1.2 Key schedule and HKDF

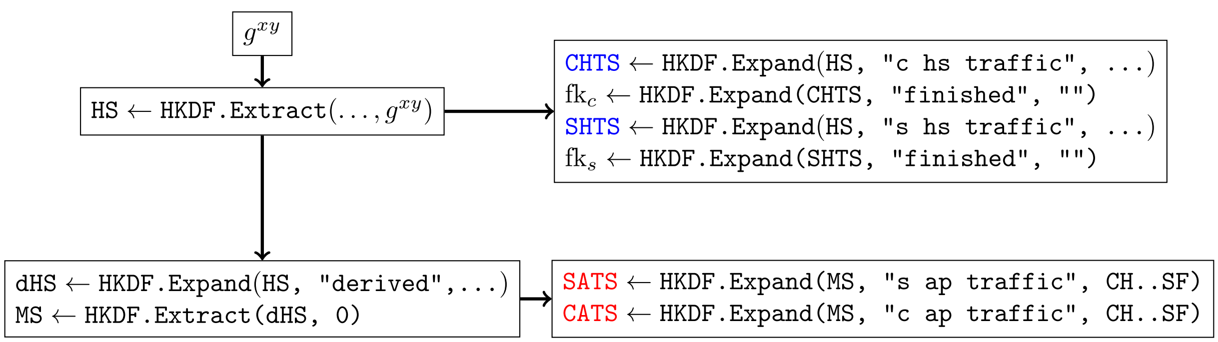

Figure 2.3: HKDF in TLS 1.3 key schedule

TLS 1.3 uses the HMAC-based Key Derivation Function (HKDF) [62] to derive cryptographic keys from input keying material (Figure 2.3). Using an extract-and-expand KDF is necessary because the n-bit shared secret obtained from public-key cryptography (e.g. g^{xy} in a Diffie-Hellman key exchange) is not a uniformly random n-bit string [61]. Instead, it typically contains fewer bits of *computational* entropy[2].

In TLS 1.3, the function `HKDF.Extract` serves as an entropy extractor that concentrates the available randomness in the input keying material. Another critical function of HKDFis to derive computationally independent keys via `HKDF.Expand`. As illustrated in Figure 2.1, all handshake messages following `ServerHello`, as well as all application data, are encrypted using different symmetric keys. Specifically, handshake messages are protected

[2] As stated in [61], an adversary with knowledge of g^x and g^y can deterministically compute g^{xy}, implying that the statistical entropy of g^{xy} given g^x and g^y is zero.

using a pair of handshake traffic keys—client handshake traffic secret (CHTS) and server handshake traffic secret (SHTS)—while application data is encrypted under a separate pair of application traffic keys—client application traffic secret (CATS) and server application traffic secret (SATS). Additionally, there is a unique symmetric key for each party to produce the HMAC tag used in their respective `Finished` messages. According to the HKDF model described in [63, 61], each of these keys—CATS, SATS, CHTS, SHTS, the client finished key, and the server finished key—is computationally independent from all others.

2.1.3 Security analysis

In the same way the TLS protocol consists of the handshake sub-protocol and the application layer, TLS protocol's security is broken down into two components. Assuming that the client and the server have already established a shared secret, the security of the application layer follows the model of authenticated encryption, which is outside this thesis' scope. Here we focus on the security of the handshake sub-protocol.

The handshake protocol is primarily modeled as an authenticated key exchange [3], but there are additional nuances to consider. First, a TLS handshake produces multiple symmetric keys (CHTS, SHTS, fk_c, fk_s, CATS, SATS), and the protocol would be considered insecure if any of these keys can be compromised. Second, an adversary can execute attacks across multiple connections involving any number of parties, and we need to account for the possibility that the adversary may compromise any long-term secret keys. Resisting replays attacks and forward secrecy both fall under the second consideration.

The security model that is based on authenticated key exchange but also includes all considerations mentioned above is called multi-stage security [32, 43], whose primary objective is *key indistinguishability* [13] for all keys derived while under active attacks. Under this model, a series of works [26, 27, 28] proved that TLS 1.3 is multi-stage secure if the nonces are sufficiently long, the underlying hash function is collision resistant, the signature and HMAC are existentially unforgeable (EUF-CMA), the Diffie-Hellman key exchange is secure (PRF-ODH [12]), and the KDF is a secure PRF.

Theorem 2.1.1 (Multi-Stage security of `TLS1.3-full-1RTT` [28])**.** *For any efficient adversary $\mathcal{A}$ against the Multi-Stage security there exist efficient algorithms $\mathcal{B}_1, \ldots, \mathcal{B}_7$* such that

$$
\mathit{Adv}^{\mathit{multi\text{-}stage}}_{\mathit{TLS1.3}}(\mathcal{A}) \leq \frac{n_s^2}{2^{|nonce|}} + \mathit{Adv}^{COLL}_{H}(\mathcal{B}_1) + 2n_s \left(\begin{array}{c} \mathit{Adv}^{COLL}_{H}(\mathcal{B}_1) + n_u \cdot \mathit{Adv}^{EUF\text{-}CMA}_{SIG}(\mathcal{B}_2) \\ + n_s \left(\begin{array}{l} \mathit{Adv}^{dual\text{-}snPRF\text{-}ODH}_{HKDS.Extract,ECDHE}(\mathcal{B}_3) \\ + \mathit{Adv}^{PRF}_{HKDF.Expand}(\mathcal{B}_4) \\ + 2 \cdot \mathit{Adv}^{PRF}_{HKDF.Expand}(\mathcal{B}_5) \\ + \mathit{Adv}^{PRF}_{HKDF.Extract}(\mathcal{B}_6) \\ + \mathit{Adv}^{PRF}_{HKDF.Expand}(\mathcal{B}_7) \end{array} \right) \end{array} \right)
$$

where n_s is the maximum number of sessions and n_u is the maximum number of users.

With the transition to PQ-TLS, we need to replace ephemeral Diffie-Hellman key exchange with post-quantum KEMs. There is unfortunately no formal analysis for TLS 1.3 using KEM instead of Diffie-Hellman for key exchange, although we speculate that the security results from Section 2.2.2 can be retrofitted to replace the PRF-ODH assumption with IND-1CCA assumption. We will discuss the possibility of using passively secure KEM for ephemeral key exchange in Section 3.4.

2.2 KEMTLS

KEMTLS [90, 44, 91], first proposed by Schwabe, Stebila, and Wiggers in 2020, is a variation on TLS 1.3 that uses key encapsulation mechanism (KEM) instead of digital signature for peer authentication. KEMTLS is motivatd by the observation that post-quantum signature candidates generally have larger public key and signature than post-quantum KEM candidates' public key and ciphertext, so by using KEMs instead of signatures, one can obtain significant savings in communication sizes.

Section 2.2.1 will introduce the KEMTLS handshake workflow. Section 2.2.2 will review the security properties of KEMTLS and compare them with TLS 1.3.

2.2.1 KEMTLS handshake

Figure 2.4 illustrates a KEMTLS handshake with unilateral authentication. Like TLS 1.3's handshake (Figure 2.1), KEMTLS handshake can be cleanly divided into a key exchange phases where the two parties establish an ephemeral shared secret, and an authentication phase where the server proves its identity to the client. However, KEMTLS' authentication flow diverges significantly from TLS 1.3's authentication flow, and there are subtle downstream differences. We will discuss them in detail in this section.

Ephemeral key exchange. Like TLS 1.3, a KEMTLS handshake begins with establishing a TCP stream and client/server exchanging `ClientHello` and `ServerHello` in this order. Client first picks a KEM scheme $\texttt{KEM}_e$ for ephemeral key exchange, then generates the ephemeral keypair $(\texttt{pk}_e, \texttt{sk}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.KeyGen}()$. The ephemeral public key $\texttt{pk}_e$ is sent to the server in `ClientHello`, which also includes a 32-byte client nonce $r_c \in \{0,1\}^{256}$, client's supported cipher suites, and other parameters not crucial to the workflow of KEMTLS. If server accepts the parameters in `ClientHello`, then it encapsulates a random secret $(\texttt{ct}_e, \texttt{ss}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.Encap}(\texttt{pk}_e)$. The ciphertext $\texttt{ct}_e$ is included in `ServerHello` sent to the client, alongside server's random nonce $r_s \in \{0,1\}^{256}$ and other parameters. Finally, after receiving `ServerHello`, client recovers the ephemeral shared secret by decapsulating the ciphertext $\texttt{ss}_e \leftarrow \texttt{KEM}_e\texttt{.Decap}(\texttt{sk}_e, \texttt{ct}_e)$.

At this stage, client and server have established an ephemeral shared secret $\texttt{ss}_e$ and will update their key schedule synchronously. Specifically, each extracts a handshake secret (HS) from $\texttt{ss}_e$ using `HKDF.Extract`, then derive client handshake traffic secret (CHTS), server handshake traffic secret (SHTS), and derived handshake secret (dHS) using `HKDF.Expand` under different labels. CHTS and SHTS will be used to encrypt subsequent handshake messages from the client and the server respectively. dHS on the other hand serves as the secret state of the key schedule and will participate in subsequent key schedule update. Figure 2.4 used color-coding and subscripts to clarify which handshake message is encrypted and under which secret.

(Implicit) Server authentication. Like TLS 1.3, the authentication phase begins with server sending `Certificate`, but the workflow diverges afterwards. With KEMTLS, server's long-term public key $\texttt{pk}_s$ is a KEM encapsulation key. To prove server's possession of the corresponding decapsulation key, client uses server's long-term public key to encapsulate a secret $(\texttt{ct}_s, \texttt{ss}_s) \xleftarrow{\$} \texttt{KEM}_s\texttt{.Encap}(\texttt{pk}_s)$, then sends the ciphertext in a handshake message $\texttt{KemCiphertext}[\texttt{ct}_s]$. After receiving $\texttt{KemCiphertext}[\texttt{ct}_s]$, server recovers the authentication secret using its long-term secret key $\texttt{ss}_s \leftarrow \texttt{KEM}_s\texttt{.Decap}(\texttt{sk}_s, \texttt{ct}_s)$.

After this round of messages, the two parties have obtained another shared secret $\texttt{ss}_s$ and will perform a second update to the key schedule. First, the (unauthenticated) handshake secret (HS) will be upgraded to an authenticated handshake secret (AHS) by

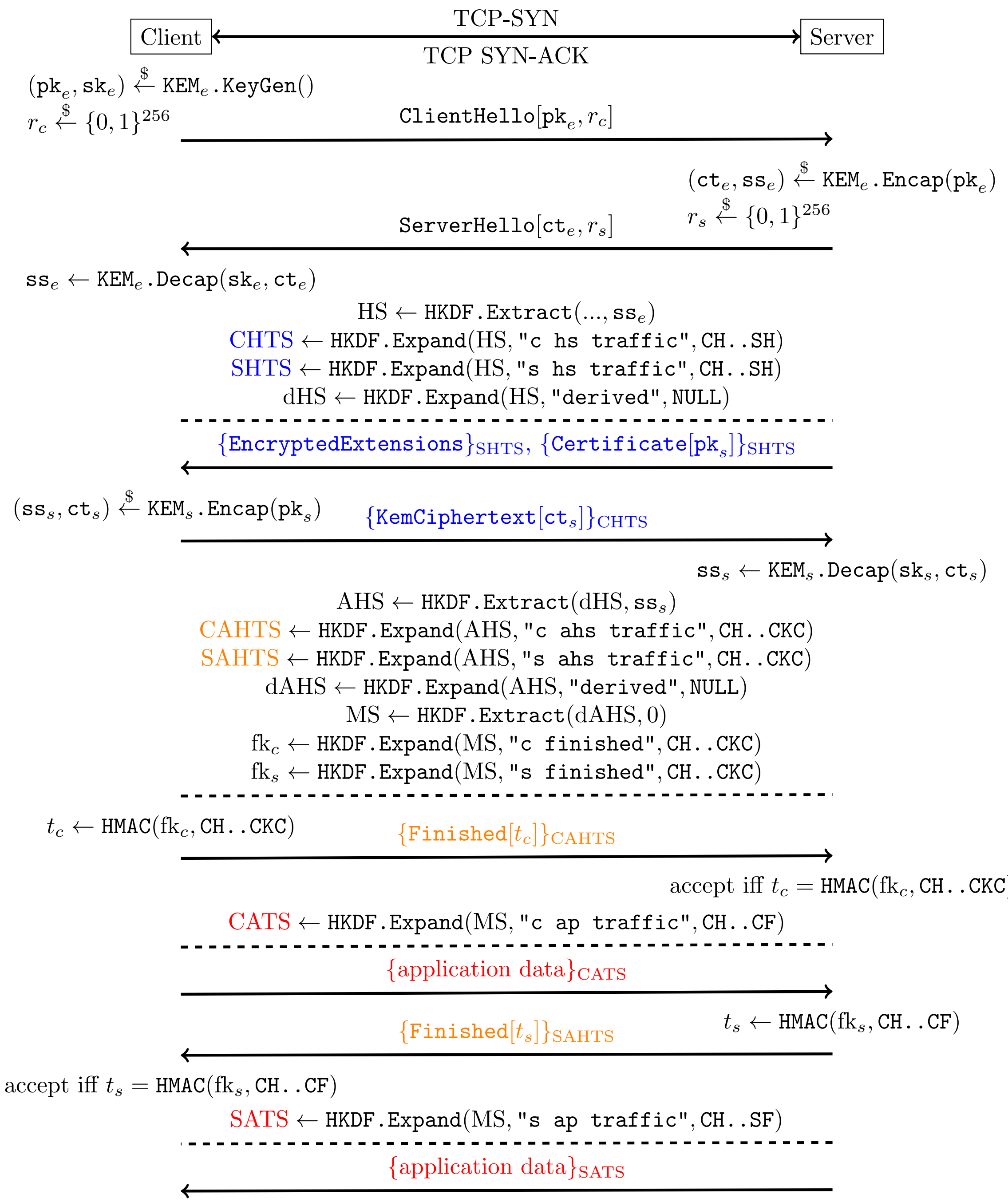

Figure 2.4: KEMTLS handshake with unilateral authentication

extracting randomness from dHS and $\mathtt{ss}_s$. From AHS, the handshake traffic secrets will be upgraded to authenticated handshake traffic secrets (CAHTS, SAHTS). The key schedule's secret state is updated from AHS, then used to produce the Master Secret (MS). Last but not least, the `Finished` keys $\mathrm{fk}_c, \mathrm{fk}_s$ are derived from MS.

Key confirmation/explicit authentication. The final round of handshake message includes client and server exchanging `Finished`. Like TLS 1.3, the `Finished` message contains an HMAC tag over the handshake transcript up to the point when the message is sent. Also like TLS 1.3, each party can derive the appropriate application traffic keys (CATS, SATS) and start sending application data right after sending its own `Finished`. On the other hand, unlike TLS 1.3, the order of "who first sends `Finished` and application" is flipped: where in TLS 1.3, server sends `Finished` before client, in KEMTLS, client sends `Finished` before server. This means that from client's perspective, TLS 1.3 and KEMTLS handshake both require 1-RTT. From server's perspective, the number of round trips required before sending application data increased, but as [90] pointed out, in most TLS applications, client sends the first round of application data, so the increase of round trips for server is likely irrelevant to the latency of the application. The flipped order of `Finished` also has security implications, since client will be sending its first round of application data before having received server's `Finished`. We will discuss them in details in Section 2.2.2.

2.2.2 KEMTLS security

KEMTLS has identical security goals to TLS 1.3. Analysis of KEMTLS' security thus follows the same framework as the formal analysis of TLS 1.3's security described in Section 2.1.3. A "pen-and-paper" proof of KEMTLS security properties is first presented in [90]. Later, a computer-verified symbolic analysis of KEMTLS was conducted using Tamarin [18, 72]. At a high level, KEMTLS achieves confidentiality, integrity, authenticity, forward secrecy, and resistance to replays if the client/server nonces are sufficiently large, the ephemeral key exchange KEM is IND-1CCA secure, the authentication KEM is IND-CCA secure, the chosen hash function is collision resistant, the HKDF is a secure PRF, and the AEAD is IND-CCA secure. These properties are formally stated by Theorem 2.2.1:

Theorem 2.2.1 (Multi-stage security of KEMTLS [90])**.** *Let n_s be the number of sessions and n_u be the number of parties. The advantage of any probabilistic polynomial-time* *adversary $\mathcal{A}$ in breaking the multi-stage security of KEMTLS is bounded by:*

$$\mathtt{Adv}_{KEMTLS}^{\mathtt{MultiStage}}(\mathcal{A}) \leq \frac{n_s^2}{2^{|nonce|}} + \epsilon_H^{COLL} + 6n_s \cdot \left(\begin{array}{l} n_s \left(\begin{array}{l} \epsilon_{\mathit{KEM}_e}^{\mathit{IND\text{-}1CCA}} + \epsilon_{\mathtt{HKDF.Extract}}^{PRF} \\ + 2\epsilon_{\mathtt{HKDF.Exract}}^{\mathit{dual\text{-}PRF}} + 3\epsilon_{\mathtt{HKDF.Expand}}^{PRF} \\ + \epsilon_{HMAC}^{EUF\text{-}CMA} \end{array} \right) \\ + n_u \left(\begin{array}{l} \epsilon_{\mathit{KEM}_s}^{\mathit{IND\text{-}CCA}} + 2\epsilon_{\mathtt{HKDF.Extract}}^{\mathit{dual\text{-}PRF}} \\ + 2\epsilon_{\mathtt{HKDF.Expand}}^{PRF} + \epsilon_{HMAC}^{EUF\text{-}CMA} \end{array} \right) \end{array} \right)$$

A note on implicit authentication. In TLS 1.3, client will not receive server's application data nor send its own application data until after it has received and validated server's `Finished`, which means that all application data exchanged are explicitly authenticated. In KEMTLS, however, client can start sending application data after sending its `Finished` but before receiving server's `Finished`. The application data sent before receiving server's `Finished` are encrypted under client's application traffic key, which contains entropy from the authentication secret $\mathtt{ss}_s$. This means that, if $\mathtt{KEM}_s$ is IND-CCA secure, then no party without the long-term seret key $\mathtt{sk}_s$ can learn anything about the application traffic keys. In other words, the early application data is **implicitly authenticated**. While this provides less security than explicitly authenticated application data, to this day there is no known attack exploiting the implicit authentication. In addition, explicit authentication can be achieved with just one more round trip, at which time KEMTLS becomes as secure as TLS 1.3 using comparable cryptographic primitives.

Chapter 3

Faster ephemeral key exchange using the encrypt-then-MAC KEM transformation

INDistinguishability under (adaptive) **C**hosen **C**iphertext **A**ttacks (IND-CCA) is the desired level of security for a key encapsulation mechanism. Intuitively, IND-CCA security requires that the shared secret from a random encapsulation cannot be distinguished from random bit string of equal length with non-negligible probability, even when the adversary has access to a decapsulation oracle. Theoretically speaking, the number of decapsulation queries is bounded by the requirement that the adversary is "efficient": that is, the adversary, classical or quantum, can only have polynomial time complexity. In practice, NIST's PQC standardization project evaluates a candidate KEM's CCA security under the upper bound of 2^{64} decapsulation queries [74].

However, within the context of an TLS 1.3 (and KEMTLS) key exchange, IND-CCA security is overkill. This is because TLS 1.3 is designed with forward secrecy, which requires that the keypair $(\texttt{pk}_e, \texttt{sk}_e) \xleftarrow{\$} \texttt{KEM}_e.\texttt{KeyGen}()$ exchanged in `ClientHello` and `ServerHello` be freshly generated at every session. Consequently, in a correct TLS 1.3 implementation, each private key will only be used to decapsulate at most one ciphertext. This naturally defines an alternative security definition called IND-1CCA, which is identical to IND-CCA except that the decapsulation oracle will answer at most one query. A similar notion of "security under one query" for Diffie-Hellman-style key exchange protocol called `snPRF-ODH` is defined in [28] and used in the security reduction of TLS 1.3 handshake protocol.

On the other hand, using IND-1CCA security brings meaningful performance improvements. A popular strategy for building an IND-CCA secure KEM is to start with an IND-CPA secure public key encryption scheme (PKE), then apply some variation of the Fujisaki-Okamoto transformation [34, 35, 47]. The key techniques for achieving chosen-ciphertext security in are *de-randomization* and *re-encryption*. While there is some contention on whether *de-randomiation* hurts security [6] (which indirectly hurts performance because the same security level may require larger parameters), there is no doubt that *re-encryption* hurts performance. The performance penalty of re-encryption is especially pronounced with many lattice-based cryptosystems whose CPA-secure encryption sub-routine is much slower than the CPA-secure decryption sub-routine. As we will show in the following section, IND-1CCA constructions can preserve randomization of the underlying PKE while not requiring re-encryption.

In Section 3.1 we will present an original IND-1CCA construction that transforms a CPA-secure PKE into an IND-1CCA secure KEM. Section 3.2 describes an instantiation of our transformation using the CPA sub-routines of ML-KEM. Section 3.3 presents the performance of our instantiation compared to ML-KEM. Finally, Section 3.4 surveys and discusses related works.

3.1 The encrypt-then-MAC transformation

Let $\mathcal{B}^*$ denote the set of finite bit strings. Let $\mathcal{K}_{\texttt{KEM}}$ denote the set of all possible shared secrets. Let $(\texttt{KeyGen}_{\texttt{PKE}}, \texttt{Enc}_{\texttt{PKE}}, \texttt{Dec}_{\texttt{PKE}})$ be a PKE defined over message space $\mathcal{M}_{\texttt{PKE}}$ and ciphertext space $\mathcal{C}_{\texttt{PKE}}$. Let $\texttt{MAC} : \mathcal{K}_{\texttt{MAC}} \times \mathcal{B}^* \to \mathcal{T}$ be a MAC over key space $\mathcal{K}_{\texttt{MAC}}$ and tag space $\mathcal{T}$. Let $G : \mathcal{B}^* \to \mathcal{K}_{\texttt{MAC}}, H : \mathcal{B}^* \to \mathcal{K}_{\texttt{KEM}}$ be hash functions. The encrypt-then-MAC transformation $\texttt{ETM}[\texttt{PKE}, \texttt{MAC}, G, H]$ constructs a KEM $(\texttt{KeyGen}_{\texttt{ETM}}, \texttt{Encap}_{\texttt{ETM}}, \texttt{Decap}_{\texttt{ETM}})$ (Figure 3.1). Definitions of these cryptographic primitives can be found in Appendix B.

We chose to construct $\texttt{KEM}_{\texttt{ETM}}$ using implicit rejection $K \leftarrow H(s, c)$: on invalid ciphertexts, the decapsulation routine returns a fake shared secret that depends on the ciphertext and some secret values, though choosing to use explicit rejection should not impact the security of the KEM. In addition, because the underlying PKE can be randomized, the shared secret $K \leftarrow H(m, c)$ must depend on both the plaintext and the ciphertext. According to [22, 47], if the input PKE is *rigid* (i.e. $m = \texttt{Dec}(\texttt{sk}, c)$ if and only if $c = \texttt{Enc}(\texttt{pk}, m)$, such as with RSA), then the shared secret may be derived from the plaintext alone $K \leftarrow H(m)$.

The CCA security of $\texttt{KEM}_{\texttt{ETM}}$ can be intuitively argued through an adversary's inability to learn additional information from the decapsulation oracle. For an adversary A to produce a valid tag for some unauthenticated ciphertext c', it must either know the correct

$\texttt{KeyGen}_{\texttt{ETM}}()$

$(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}_{\texttt{PKE}}()$
$s \xleftarrow{\$} \mathcal{M}_{\texttt{PKE}}$
$\texttt{sk} \leftarrow (\texttt{sk}, s)$
return $(\texttt{pk}, \texttt{sk})$

$\texttt{Encap}_{\texttt{ETM}}(\texttt{pk})$

$m \xleftarrow{\$} \mathcal{M}_{\texttt{PKE}}$
$k \leftarrow G(m)$
$c' \xleftarrow{\$} \texttt{Enc}_{\texttt{PKE}}(\texttt{pk}, m)$
$t \leftarrow \texttt{MAC}(k, c')$
$c \leftarrow (c', t)$
$K \leftarrow H(m, c)$
return (c, K)

$\texttt{Decap}_{\texttt{ETM}}(\texttt{sk}, c)$

Require: $c = (c', t)$
Require: $\texttt{sk} = (\texttt{sk}', s)$
$\hat{m} \leftarrow \texttt{Dec}_{\texttt{PKE}}(\texttt{sk}', c')$
$\hat{k} \leftarrow G(\hat{m})$
if $\texttt{MAC}(\hat{k}, c') = t$ **then**
    $K \leftarrow H(\hat{m}, c)$
else
    $K \leftarrow H(s, c)$
end if
return K

Figure 3.1: The encrypt-then-MAC KEM routines

symmetric key or produce a forgery. Under the Random Oracle Model (ROM), A cannot know the symmetric key without knowing its pre-image under the hash function G, so A must either produced c' honestly, or have broken the one-wayness of the underlying PKE. This means that the decapsulation oracle will not leak information on decryption that the adversary does not already know. We formalize the security in Theorem 3.1.1

Theorem 3.1.1. *For every IND-qCCA adversary A against $\texttt{KEM}_{\texttt{ETM}}$ making q decapsulation queries, there exists a OW-PCA adversary B against the underlying PKE making at least q decapsulation queries, and an existential forgery adversary C against the underlying MAC such that:*

$$\mathbf{Adv}^{\text{IND-CCA}}_{\text{KEM}_{\text{ETM}}}(A) \leq q \cdot \mathbf{Adv}_{\text{MAC}}(C) + 2 \cdot \mathbf{Adv}^{\text{OW-PCA}}_{\text{PKE}}(B)$$

From here, we can reduce OW-PCA security to OW-CPA security using a guessing argument: a CPA adversary can simulate a PCA oracle simply by randomly sampling from $\{0, 1\}$ as response to every plaintext-checking query. Thus, the probability that all guesses are correct and PCA adversary retains its advantage is at most 2^{-q}. In other words:

Lemma 3.1.2. *For every efficient q-query OW-PCA adversary A against some PKE, there exists an OW-CPA adversary B such that*

$$\mathbf{Adv}^{\text{OW-PCA}}_{\text{PKE}}(A) \leq 2^q \cdot \mathbf{Adv}^{\text{OW-CPA}}_{\text{PKE}}(B)$$

3.1.1 Proof of Theorem 3.1.1

We will prove Theorem 3.1.1 using a sequence of game. A summary of the sequence of games can be found in Figure 3.2 and 3.3. From a high level we made three incremental modifications to the IND-CCA game for $\texttt{KEM}_{\texttt{ETM}}$:

1. Replace the true decapsulation oracle with a simulated decapsulation oracle. The simulated decapsulation oracle does not directly decrypt the queried ciphertext. Instead it searches through the hash oracle and looks for matching queries using the plaintext-checking oracle. The true decapsulation oracle and the simulated decapsulation oracle disagree if and only if the adversary queries with a ciphertext that contains a forged MAC tag, so the adversary cannot distinguish the two games more than it can perform existential forgery against the underlying MAC.

2. Replace the pseudorandom MAC key $k^* \leftarrow G(m^*)$ with a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$. Under ROM, the adversary cannot distinguish this game from the previous one unless it queries the hash oracle with m^*, in which case a second adversary with access to a plaintext-checking oracle can win the OW-PCA game against the underlying PKE.

3. Replace the pseudorandom shared secret $K_0 \leftarrow H(m^*, c)$ with a truly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$. Similar to the point above, the adversary cannot distinguish the change more than it can break the one-wayness of the underlying PKE. Furthermore, since both K_0 and K_1 are uniformly random, no adversary can have any advantage.

A OW-PCA adversary can then simulate the modified IND-CCA game for the KEM adversary, and the advantage of the OW-PCA adversary is associated with the probability of certain behaviors of the KEM adversary.

Proof. Game 0 is the standard KEM IND-CCA game. The decapsulation oracle $\mathcal{O}^{\texttt{Decap}}$ executes the decapsulation routine using the challenge keypair and return the results faithfully. The queries made to the hash oracles $\mathcal{O}^G, \mathcal{O}^H$ are recorded to their respective tapes $\mathcal{L}^G, \mathcal{L}^H$.

Game 1 is identical to game 0 except that the true decapsulation oracle $\mathcal{O}^{\texttt{Decap}}$ is replaced with a simulated oracle $\mathcal{O}_1^{\texttt{Decap}}$. Instead of directly decrypting c' as in the decapsulation routine, the simulated oracle searches through the tape $\mathcal{L}^G$ to find a matching query $(\tilde{m}, \tilde{k})$ such that $\tilde{m}$ is the decryption of c'. The simulated oracle then uses $\tilde{k}$ to validate the tag t against c'.

$\texttt{IND-CCA}$ game for $\texttt{KEM}_{\texttt{ETM}}$

$(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}_{\texttt{ETM}}()$
$m^* \xleftarrow{\$} \mathcal{M}$
$c' \xleftarrow{\$} \texttt{Enc}_{\texttt{PKE}}(\texttt{pk}, m^*)$
$k^* \leftarrow G(m^*)$ ▷ Game 0-1
$k^* \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$ ▷ Game 2-3
$t \leftarrow \texttt{MAC}(k^*, c')$
$c^* \leftarrow (c', t)$
$K_0 \leftarrow H(m^*, c^*)$ ▷ Game 0-2
$K_0 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$ ▷ Game 3
$K_1 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$
$b \xleftarrow{\$} \{0, 1\}$
$\hat{b} \leftarrow A^{\mathcal{O}^{\texttt{Decap}}}(\texttt{pk}, c^*, K_b)$ ▷ Game 0
$\hat{b} \leftarrow A^{\mathcal{O}_1^{\texttt{Decap}}}(\texttt{pk}, c^*, K_b)$ ▷ Game 1-3
return $[\![\hat{b} = b]\!]$

Hash oracle $\mathcal{O}^G(m)$

if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ **then**
  return $\tilde{k}$
end if
$k \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$
$\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$
return k

Decap oracle $\mathcal{O}^{\texttt{Decap}}(c)$

$(c', t) \leftarrow c$
$\hat{m} = \texttt{Dec}_{\texttt{PKE}}(\texttt{sk}', c')$
$\hat{k} \leftarrow G(\hat{m})$
if $\texttt{MAC}(\hat{k}, c') = t$ **then**
  $K \leftarrow H(\hat{m}, c)$
else
  $K \leftarrow H(z, c)$
end if
return K

$\mathcal{O}_1^{\texttt{Decap}}(c)$

$(c', t) \leftarrow c$
if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = \texttt{Dec}_{\texttt{PKE}}(\texttt{sk}', c') \wedge \texttt{MAC}(\tilde{k}, c') = t$ **then**
  $K \leftarrow H(\tilde{m}, c)$
else
  $K \leftarrow H(z, c)$
end if
return K

$\mathcal{O}^H(m, c)$

if $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ **then**
  return $\tilde{K}$
end if
$K \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$
$\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$
return K

Figure 3.2: Sequence of games in the proof of Theorem 3.1.1

$B(\mathtt{pk}, c'^*)$

$z \xleftarrow{\$} \mathcal{M}$
$k \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$
$t \leftarrow \mathtt{MAC}(k, c'^*)$
$c^* \leftarrow (c'^*, t)$
$K \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
$\hat{b} \leftarrow A^{\mathcal{O}_B^{\mathtt{Decap}}, \mathcal{O}_B^G, \mathcal{O}_B^H}(\mathtt{pk}, c^*, K)$
if $\mathtt{ABORT}(m)$ **then**
    return m
end if

$\mathcal{O}_B^H(m, c)$

if $\mathcal{O}^{\mathtt{PCO}}(m, c'^*) = 1$ **then**
    $\mathtt{ABORT}(m)$
end if
if $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ **then**
    return $\tilde{K}$
end if
$K \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
$\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$
return K

$\mathcal{O}_B^{\mathtt{Decap}}(c)$

$(c', t) \leftarrow c$
if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \mathcal{O}^{\mathtt{PCO}}(\tilde{m}, c') = 1 \wedge \mathtt{MAC}(\tilde{k}, c') = t$ **then**
    $K \leftarrow H(\tilde{m}, c)$
else
    $K \leftarrow H(z, c)$
end if
return K

$\mathcal{O}_B^G(m)$

if $\mathcal{O}^{\mathtt{PCO}}(m, c'^*) = 1$ **then**
    $\mathtt{ABORT}(m)$
end if
if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ **then**
    return $\tilde{k}$
end if
$k \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$
$\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$
return k

Figure 3.3: OW-PCA adversary B simulates game 3 for IND-CCA adversary A in the proof for Theorem 3.1.1

If the simulated oracle accepts the queried ciphertext as valid, then there is a matching query that also validates the tag, which means that the queried ciphertext is honestly generated. Therefore, the true oracle must also accept the queried ciphertext. On the other hand, if the true oracle rejects the queried ciphertext, then the tag is simply invalid under the MAC key $k = G(\texttt{Dec}(\texttt{sk}', c'))$. Therefore, there could not have been a matching query that also validates the tag, and the simulated oracle must also rejects the queried ciphertext.

This means that from the adversary A's perspective, game 1 and game 0 differ only when the true oracle accepts while the simulated oracle rejects, which means that t is a valid tag for c' under $k = G(\texttt{Dec}(\texttt{sk}', c'))$, but k has never been queried. Under the random oracle model, such k is a uniformly random sample of $\mathcal{K}_{\texttt{MAC}}$ that the adversary does not know, so for A to produce a valid tag is to produce a forgery against the MAC under an unknown and uniformly random key. Therefore, we can bound the probability that the true decapsulation oracle disagrees with the simulated oracle by the probability that some MAC adversary produces a forgery:

$$P\left[\mathcal{O}^{\texttt{Decap}}(c) \neq \mathcal{O}_1^{\texttt{Decap}}(c)\right] \leq \texttt{Adv}_{\texttt{MAC}}(C).$$

Across all q decapsulation queries, the probability that at least one query is a forgery is thus at most $q \cdot P\left[\mathcal{O}^{\texttt{Decap}}(c) \neq \mathcal{O}_1^{\texttt{Decap}}(c)\right]$. By the difference lemma:

$$|\texttt{Adv}_{G_0}(A) - \texttt{Adv}_{G_1}(A)| \leq q \cdot \texttt{Adv}_{\texttt{MAC}}(C).$$

Game 2 is identical to game 1, except that the challenger samples a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$ instead of deriving it from m^*. From A's perspective the two games are indistinguishable, unless A queries G with the value of m^*. Denote the probability that A queries G with m^* by $P[\texttt{QUERY G}]$, then:

$$|\texttt{Adv}_{G_1}(A) - \texttt{Adv}_{G_2}(A)| \leq P\left[\texttt{QUERY G}\right].$$

Game 3 is identical to game 2, except that the challenger samples a uniformly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$ instead of deriving it from m^* and t. From A's perspective the two games are indistinguishable, unless A queries H with $(m^*, \cdot)$. Denote the probability that A queries H with $(m^*, \cdot)$ by $P[\texttt{QUERY H}]$, then:

$$|\texttt{Adv}_{G_2}(A) - \texttt{Adv}_{G_3}(A)| \leq P\left[\texttt{QUERY H}\right].$$

Since in game 3, both K_0 and K_1 are uniformly random and independent of all other variables, no adversary can have any advantage: $\mathtt{Adv}_{G_3}(A) = 0$.

We will bound $P[\mathtt{QUERY\ G}]$ and $P[\mathtt{QUERY\ H}]$ by constructing a OW-PCA adversary B against the underlying PKE that uses A as a sub-routine. B's behaviors are summarized in Figure 3.3.

B simulates game 3 for A: upon receiving the public key $\mathtt{pk}$ and challenge encryption c'^*, B samples random MAC key and session key to produce the challenge encapsulation, then feeds it to A. When simulating the decapsulation oracle, B uses the plaintext-checking oracle to look for matching queries in $\mathcal{L}^G$. When simulating the hash oracles, B uses the plaintext-checking oracle to detect when $m^* = \mathtt{Dec}(\mathtt{sk}', c'^\star)$ has been queried. When m^* is queried, B terminates A and returns m^* to win the OW-PCA game. In other words:

$$P[\mathtt{QUERY\ G}] \leq \mathtt{Adv}_{\mathtt{PKE}}^{\mathtt{OW\text{-}PCA}}(B),$$
$$P[\mathtt{QUERY\ H}] \leq \mathtt{Adv}_{\mathtt{PKE}}^{\mathtt{OW\text{-}PCA}}(B).$$

Combining all equations above produce the desired security bound. □

3.2 Practical instantiation with ML-KEM

$\mathtt{KEM}^{\not\perp}_m.\mathtt{KeyGen}()$

$(\mathtt{pk}, \mathtt{sk}') \xleftarrow{\$} \mathtt{PKE.KeyGen}()$
$z \xleftarrow{\$} \mathcal{M}$
$\mathtt{sk} \leftarrow (\mathtt{sk}', \mathtt{pk}, z)$
return $(\mathtt{pk}, \mathtt{sk})$

$\mathtt{KEM}^{\not\perp}_m.\mathtt{Encap}(\mathtt{pk})$

$m \xleftarrow{\$} \mathcal{M}$
$r \leftarrow G(m)$
$c \leftarrow \mathtt{PKE.Enc}(\mathtt{pk}, m, r)$
$K \leftarrow H(m)$
return (c, K)

$\mathtt{KEM}^{\not\perp}_m.\mathtt{Decap}(\mathtt{sk}, c)$

$\hat{m} \leftarrow \mathtt{PKE.Dec}(\mathtt{sk}', c)$
$\hat{r} \leftarrow G(m)$
$\hat{c} \leftarrow \mathtt{PKE.Enc}(\mathtt{pk}, \hat{m}, \hat{r})$
if $\hat{c} = c$ **then**
    $K \leftarrow H(\hat{m})$
else
    $K \leftarrow H(z, c)$
end if
return K

Figure 3.4: The $\mathtt{KEM}^{\not\perp}_m$ variant of FO transformation [47] is used in ML-KEM

ML-KEM is an IND-CCA secure KEM standardized by NIST in FIPS 203 [78]. The CCA security of ML-KEM is achieved in two steps. The first step constructs a PKE

(Figure C.1) whose IND-CPA security reduces to the conjectured intractability of the Decisional Module Learning with Error (MLWE) problem. Then the $\mathtt{KEM}_m^{\not\perp}$ variant of the FO transformation (Figure 3.4) is applied to convert the IND-CPA secure PKE into an IND-CCA secure KEM. An algorithmic description of ML-KEM can be found in Appendix C.

3.2.1 One-time ML-KEM (OT-ML-KEM)

We apply the encrypt-then-MAC transformation to the K-PKE routines of ML-KEM and call the transformed KEM *one-time ML-KEM (OT-ML-KEM)* (Figure 3.5). The key generation routines are identical between OT-ML-KEM and ML-KEM. The OT-ML-KEM encapsulation and decapsulation routines make additional use a MAC and a KDF.

`OT-ML-KEM.KeyGen()`

$z \xleftarrow{\$} \{0,1\}^{256}$
$(\mathtt{pk}, \mathtt{sk}') \xleftarrow{\$} \mathtt{K\text{-}PKE.KeyGen}()$
$h \leftarrow H(\mathtt{pk})$
$\mathtt{sk} \leftarrow (sk' \| \mathtt{pk} \| h \| z)$
return $(\mathtt{pk}, \mathtt{sk})$

`OT-ML-KEM.Encap(pk)`

$m \xleftarrow{\$} \{0,1\}^{256}$
$(\overline{K}, k) \leftarrow G(m \| H(\mathtt{pk}))$
$c' \xleftarrow{\$} \mathtt{K\text{-}PKE.Enc}(\mathtt{pk}, m)$
$t \leftarrow \mathtt{MAC}(k, c')$
$K \leftarrow \mathtt{KDF}(\overline{K} \| t)$
$c \leftarrow (c', t)$
return (c, K)

`OT-ML-KEM.Decap(sk, c)`

Require: Secret key $\mathtt{sk} = (\mathtt{sk}' \| \mathtt{pk} \| h \| z)$
Require: Ciphertext $c = (c' \| t)$
$(\mathtt{sk}', \mathtt{pk}, h, z) \leftarrow \mathtt{sk}$
$(c', t) \leftarrow c$
$\hat{m} \leftarrow \mathtt{K\text{-}PKE.Dec}(\mathtt{sk}', c')$
$(\overline{K}, \hat{k}) \leftarrow G(\hat{m} \| h)$
$\hat{t} \leftarrow \mathtt{MAC}(\hat{k}, c')$
if $\hat{t} = t$ **then**
  $K \leftarrow \mathtt{KDF}(\overline{K} \| t)$
else
  $K \leftarrow \mathtt{KDF}(z \| c)$
end if
return K

Figure 3.5: OT-ML-KEM applies the encrypt-then-MAC transformation to the K-PKE sub-routines of ML-KEM. G is SHA3-512, H is SHA3-256, $\mathtt{KDF}$ is Shake256

For performance and practical security, OT-ML-KEM makes two modifications to the encrypt-then-MAC transformation:

- *Hashing public key into MAC key and shared secret.* The public key goes into deriving both the MAC key and the shared secret for the same two reasons why Kyber [10] hashes its public key into the pseudorandom coin and the shared secret. First, we want the KEM to be *contributory*, meaning both parties in a two-party key exchange have inputs into the shared secret. Second, we want to enhance the KEM's multi-user security: if the MAC key is derived from the PKE plaintext alone, then an adversary can pre-compute a large dictionary mapping MAC keys to the pre-image PKE plaintext. Upon receiving some ciphertext, this adversary can search through this key-plaintext dictionary and recover the decryption. Hashing the public key into the MAC key prevents an adversary from using a single lookup table against multiple keypairs.

- *Hashing the MAC tag instead of the entire ciphertext.* Because we allowed the PKE to be randomized, the shared secret must have inputs from both the plaintext and the ciphertext. However, we find it unnecessary to hash the entire ciphertext. Instead, if the MAC is modeled as a secure pseudorandom function (PRF), then the MAC tag functions as a (keyed) hash of the ciphertext, so we use the much shorter MAC tag as a substitute.

Compared to ML-KEM, OT-ML-KEM makes the following trade-offs:

- OT-ML-KEM's encapsulation is slightly slower because of the additional steps of computing the MAC tag
- OT-ML-KEM ciphertext size increases by the size of the MAC tag.
- OT-ML-KEM replaces *re-encryption* with computing the MAC tag.
- As the name suggests, each OT-ML-KEM keypair can only be used once.

3.2.2 Choosing a message authentication code

We instantiated OT-ML-KEM with a diverse set of MAC designs, including Poly1305 [5], GMAC [70], CMAC [9, 52], and KMAC [41]. All four schemes are parameterized with 256-bit key and 128-bit tag (except for KMAC, see paragraph below) to ensure that MAC keys are at least as hard to guess has the underlying PKE plaintext.

Poly1305 and GMAC are both Carter-Wegman style MAC [108] that compute the tag using finite field arithmetics. Each scheme operates by breaking the message into blocks that can then be parsed into finite field elements. The tag is computed by evaluating a polynomial whose coefficients are the message blocks and whose indeterminate is the secret key. Poly1305 operates in the prime field $GF(2^{130} - 5)$ whereas GMAC operates in the binary field $GF(2^{128})$.

CMAC is based on the CBC-MAC with the block cipher instantiated from AES-256. To compute a CMAC tag, the message is first broke into 128-bit blocks with appropriate padding. Each block is first XORed with the previous block's output, then encrypted under AES using the symmetric key. The final output is XORed with a sub key derived from the symmetric key, before being encrypted for one last time.

KMAC is based on the SHA-3 family of sponge functions [76]. We chose KMAC-256, which uses Shake256 as the underlying extendable output functions. KMAC is the only MAC to support variable key and tag length, though we fixed the key length at 256 bits. On the other hand, we chose the appropriate tag length for the corresponding security level: when instantiated with ML-KEM-512, the tag length is 128 bits; with ML-KEM-768, the tag length is 192 bits; with ML-KEM-1024, the tag length is 256 bits.

3.3 Performance of OT-ML-KEM

We implemented OT-ML-KEM by extending the reference implementation[1] and tested its performance on a desktop processor. All C code is compiled with `GCC 11.4.1` and `OpenSSL 3.0.8`. All binaries are executed on an AWS c7a.medium instance[2] with AMD EPYC 9R14 CPU at 3.7 GHz and 1 GB of RAM in the `us-west-2` region. Source code is available on GitHub[3].

3.3.1 MAC performance

We isolated the performance of OpenSSL's MAC implementations by measuring the CPU cycles consumed to compute a tag on random inputs whose sizes correspond to the ciphertext sizes of ML-KEM at NIST levels 1, 3, and 5. The median CPU cycle counts in 10,000 rounds of testing are summarized in Table 3.1.

[1] https://github.com/pq-crystals/kyber

[2] We chose the C7 family of EC2 instance because each vCPU in this class of VMs corresponds directly to a physical CPU core instead of a thread on a physical core

[3] https://github.com/xuganyu96/faster-generic-ind-cca-kem-using-encrypt-then-mac/tree/tches2025-volume2

Remark. In OpenSSL's implementation, Poly1305 is an one-time secure MAC. OpenSSL does not have a standalone GMAC; instead we used the AEAD `AES-256-GCM` but with all data treated as "associated data". CMAC and KMAC-256 are always many-time secure.

MAC	**NIST level 1**		**NIST level 3**		**NIST level 5**	
	input size	768 B	input size	1088 B	input size	1568 B
Poly1305		909		961		1065
GMAC		3899		4827		4055
CMAC		6291		7588		8735
KMAC-256		6373		9697		11647

Table 3.1: MAC performance (cycles/tick)

3.3.2 KEM routines performance

We ran each KEM routines 10,000 times and measured their CPU cycles. Table 3.2 summarizes the results. Compared to the $\mathtt{KEM}_m^{\not\perp}$ variant of Fujisaki-Okamoto transformed used in ML-KEM, the encrypt-then-MAC transformation achieves massive CPU cycle count reduction in decapsulation while incurring only a minimal increase of encapsulation cycle count and ciphertext size. Since `K-PKE.Enc` carries significantly more computational complexity than `K-PKE.Dec` or any MAC we chose, the performance advantage of the encrypt-then-MAC transformation over the $\mathtt{KEM}_m^{\not\perp}$ transformation is dominated by the runtime saving gained from replacing *re-encryption* with MAC.

3.4 Related works

Optimal Asymmetric Encryption[4] is a generic chosen-ciphertext secure PKE. Under ROM, the chosen-ciphertext security of OAEP reduces to the one-wayness of the input trapdoor permutation. Although there are trapdoor permutations with which OAEP does not achieve full IND-CCA security [95], Fujisaki et al. later proved that OAEP is IND-CCA secure when combined with the RSA trapdoor permutation [36]. However, OEAP's input requirement for a trapdoor permutation is difficult to satisfy, and no other practical instantiation saw widespread adoption to this day.

The *Fujisaki-Okamoto transformation* [34, 35] is another generic IND-CCA secure transformation. The original proposal constructs a hybrid PKE that uses the input PKE to

		KEM	Encap	Decap
NIST level 1		ML-KEM-512	91.46	121.18
`pk` size	800	OT-ML-KEM-512 + Poly1305	93.15	33.73
`sk` size	1632	OT-ML-KEM-512 + GMAC	97.36	37.72
`ct` size	768	OT-ML-KEM-512 + CMAC	99.73	40.11
`KeyGen` performance	75.94	OT-ML-KEM-512 + KMAC-256	101.00	40.74
NIST level 3		ML-KEM-768	136.40	186.44
`pk` size	1184	OT-ML-KEM-768 + Poly1305	146.40	43.31
`sk` size	2400	OT-ML-KEM-768 + GMAC	149.52	46.51
`ct` size	1088	OT-ML-KEM-768 + CMAC	153.13	49.84
`KeyGen` performance	129.89	OT-ML-KEM-768 + KMAC-256	155.21	52.41
NIST level 5		ML-KEM-1024	199.18	246.24
`pk` size	1568	OT-ML-KEM-1024 + Poly1305	205.76	51.37
`sk` size	3168	OT-ML-KEM-1024 + GMAC	208.80	54.57
`ct` size	1568	OT-ML-KEM-1024 + CMAC	213.66	59.17
`KeyGen` performance	194.92	OT-ML-KEM-1024 + KMAC-256	216.76	62.26

Table 3.2: OT-ML-KEM performance (kilocycles/tick)

encrypt a symmetric key, then uses the symmetric key to encrypt data. The CCA security of the FO-transformed PKE reduces non-tightly to the one-wayness of the input PKE and the semantic security of the input symmetric cipher. Later works [22, 47, 31, 48] tightened the security reduction, accounted for imperfect correctness, adapted the core techniques to build a KEM, and produced a security reduction under QROM. Thanks to its simplicity and proven security against classical and quantum adversaries, the FO transformation was adopted by many submissions to NIST's PQC standardization project.

The FO transformation is not perfect. Common criticism against FO transformation focuses on *re-encryption*, which is not only slows down decapsulation, but increases risk of side-channel. As demonstrated in [107, 101, 85], *re-encryption* is the entrypoint of many side-channel-assisted plaintext-checking attacks on NIST PQC candidates.

Bounded CCA security is first proposed by Cramer et al.[20], who proposed a generic black-box IND-qCCA PKE construction whose security reduces to the IND-CPA security of the underlying PKE and the existential unforgeability of the underlying one-time signature [64]. Besides being the first of its kind, Cramer's IND-qCCA transformation is also remarkable because the security reduction is proved under the standard model insteasd of the ROM. Unfortunately, this transformation is also computationally ineffiency and thus impractical.

Huguenin-Dumittan and Vaudenay [49] extended this line of work to the post-quantum

KeyGen
1: $(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}^{\texttt{PKE}}()$

Encap(pk)
1: $m \xleftarrow{\$} \mathcal{M}^{\texttt{PKE}}$
2: $c \xleftarrow{\$} \texttt{Enc}^{\texttt{PKE}}(\texttt{pk}, m)$
3: $K \leftarrow H(m, c)$
4: **return** (c, K)

Decap(sk, c)
1: $\hat{m} \leftarrow \texttt{Dec}^{\texttt{PKE}}(\texttt{sk}, c)$
2: **if** $\hat{m} = \perp$ **then**
3:     **return** $\perp$
4: **end if**
5: **return** $H(\hat{m}, c)$

Figure 3.6: T_H transformation

setting by proposing two IND-qCCA KEM constructions (respectively denoted by T_{CH} and T_H) from CPA-secure PKEs. T_{CH} achieves qCCA security using a "confirmation hash": the KEM ciphertext includes the PKE ciphertext and a hash of the PKE plaintext, and the KEM shared secret is derived from hashing the PKE plaintext. The security reduction is loose by a factor of 2^q where q is the number of decapsulation queries. T_H achieves qCCA security by deriving the shared secret from hashing both the PKE plaintext and ciphertext (Figure 3.6), so decapsulation queries with modified ciphertext will be answered with independent random values under the rancom oracle model. T_H's construction is simpler than T_{CH} and retains the original ciphertext length, butthe security reduction is looser, with a factor of q_H^{2q}, where q_H is the number of hash oracle queries, and q is the number of decapsulation queries. The authors also proved qCCA security of both transformations under quantum random oracle model (QROM).

In the same paper, the two authors also observed that in TLS 1.3, the handshake secret (and thus all subsequent secrets and keys) is derived with input from the handshake transcript, which means that the KEM public key and ciphertext, embedded in `ClientHello` and `ServerHello` respectively, are already hashed into the handshake secret. This is similar to the T_H construction where the shared secret is hashed from PKE plaintext and ciphertext, and indeed the authors proved that TLS 1.3 remains multi-stage secure even if the KEM is only CPA secure. The initial proof presented by the authors had a looseness factor of $O(q^6)$, which was later improved by Zhou et al.[111] to $O(q)$ for randomized PKEs, and $O(1)$ for rigid PKEs.

Chapter 4

Implementing PQ-TLS and KEMTLS on embedded client

To evaluate post-quantum TLS 1.3 and KEMTLS on embedded clients, we implemented post-quantum TLS 1.3 and KEMTLS on top of WolfSSL[1] and using cryptographic primitives from PQClean[2][59]. Source code for this project is available on GitHub[3]. This chapter describes how we implemented post-quantum TLS 1.3 and KEMTLS.

4.1 WolfSSL

WolfSSL is a modern open-source TLS library written in the C programming language. In addition to a complete TLS stack, WolfSSL also includes a robust cryptogrpahy library called WolfCrypt. WolfSSL is optimized for code size, memory footprint, and portability, and WolfCrypt received many industry/government certification including FIPS140-3 from the US federal government and DO-178C DAL-A for avionics. There are other TLS/SSL libraries with support for embedded devices. For example, [15] evaluated post-quantum TLS 1.2 using the open-source mbedTLS library [4]. Unfortunately mbedTLS did not support TLS 1.3 until December 2021. We chose to work with WolfSSL as it is the more popular choice amongst prior works [106, 105, 104, 40], which makes it easier to compare reults.

[1] https://www.wolfssl.com/

[2] https://github.com/PQClean/PQClean/

[3] https://github.com/xuganyu96/embedded-pqtls

[4] https://github.com/Mbed-TLS/mbedtls

4.1.1 Integrating PQC into WolfCrypt

PQClean is a repository of thoroughly tested, high-confidence C implementations of KEM and signature schemes submitted to NIST's PQC standardization project. The "clean" implementations ensure that C programming best practices such as memory safety and integer overflows are enforced and that the correctness and basic security defense such as avoiding branching on secret data are continuously tested. The project also improved developer ergonomics by applying consistent namespacing on exported symbols and a uniform set of APIs across all curated submissions.

PQClean comes with "batteries-included": it contains reusable symmetric primitives (AES, SHA-2, and SHA-3) shared across downstream implementations. The inclusion of shared symmetric primitives made integrating PQClean into WolfCrypt easier, but unfortunately it duplicates existing functionalities from WolfCrypt, which inflates the size of the firmware. In addition, PQClean's symmetric primitives exhibit different performance characteristics from WolfCrypt: for example, WolfCrypt contains an in-house implementation of ML-KEM and ML-DSA, which uses an in-house implementation of Keccak/SHA-3. WolfCrypt's own benchmark showed that on the embedded client, WolfCrypt's SHA-3 to have approximately 10% more throughput than PQClean's `fips202.c`, while WolfCrypt's ML-KEM is roughly twice as fast as the "clean" ML-KEM. In the end, we chose to disable WolfCrypt's ML-KEM/ML-DSA and use the "clean" implementations (using PQClean's symmetric primitives) across the entire project to ensure fair comparison across different schemes. We leave it for future works to incorporate machine-optimized implementations such as from `pqm4`[5] and `ml-kem-native`[6].

While we kept PQClean's symmetric primitives, we had to modify its `randombytes` API for generating random number. PQClean's existing `randombytes` implementation is stateless and asssumes the existence of a filesystem on some operating systems. While this works on a desktop environment, it does not work with baremetal embedded clients. In addition, most PQC implementations do not expose internal randomness through some `seed` parameter in the public API, so a wholesale replacement of `randombytes` by another RNG implementation will require extensive refactor of the entire project. Fortunately, WolfCrypt provides an abstraction `WC_RNG` with ready-to-go backends on both Linux and baremetal via Pico-SDK. We adapted the stateless `randombytes` API to use the stateful `WC_RNG` by adding a static `WC_RNG *` pointer in PQClean and implementing a function that sets this static pointer to a user-specified instance of `WC_RNG` (Listing 4.1). This also means that all PQClean schemes will source from the same randomness used throughout the TLS

[5] https://github.com/mupq/pqm4

[6] https://github.com/pq-code-package/mlkem-native

handshake.

```c
#ifdef HAVE_WC_RNG
#include <wolfssl/wolfcrypt/random.h>
static WC_RNG *g_rng;
void set_wc_rng(WC_RNG *input) {
    g_rng = input;
}
#endif

int randombytes(uint8_t *output, size_t n) {
    void *buf = (void *)output;
    #ifdef HAVE_WC_RNG
    (void)buf;
    return wc_RNG_GenerateBlock(g_rng, output, n);
    #endif
}
```

Listing 4.1: Use `WC_RNG` as backend for PQClean's `randombytes` API

4.1.2 Implementing post-quantum key exchange

Adding new KEM

Incorporating post-quantum KEMs into the key exchange flow of TLS 1.3 is relatively straightforward despite the API difference between a KEM and a Diffie-Hellman key exchange. In fact, WolfSSL already supports ML-KEM for the key exchange phase, which provides a template with which we can integrate HQC and IND-1CCA ML-KEM. RFC 8446 specified that each key exchange group (called `NamedGroup`) is identified with a 16-bit value, which is captured in an enum type `NamedGroup` in WolfSSL. The variants of this enum type are assigned their group point to be directly used in constructing the `key_share` extension the hello messages. The TLS state machine then uses a case-switch block to call the correct lower-level subroutines based on the variant. Figure 4.2 provides an example.

The explicit case-switch statement stands in contrast with TLS libraries implemented using languages with more sophisticated abstraction such as generics. For example, `rustls`, a TLS library written in Rust, takes advantage of Rust's traits/generics syntax, which enabled user to provide its own cryptographic backend by implementing a `CryptoProvider` struct (Listing 4.3). While the lack of ergonomic generics in the C programming language made much of WolfSSL's codebase verbose and sometimes difficult to read long case-switch

```c
static int TLSX_KeyShare_GenPqcKeyClient(WOLFSSL *ssl,
                                         KeyShareEntry* kse) {
    int ret = 0;
    switch (kse->group) {
        case WOLFSSL_ML_KEM_512:
        case WOLFSSL_ML_KEM_768:
        case WOLFSSL_ML_KEM_1024:
            ret = TLSX_KeyShare_GenMlKemKeyClient(ssl, kse);
            break;
        case HQC_128:
        case HQC_192:
        case HQC_256:
            ret = TLSX_KeyShare_GenHqcKeyClient(ssl, kse);
            break;
        case OT_ML_KEM_512:
        case OT_ML_KEM_768:
        case OT_ML_KEM_1024:
            ret = TLSX_KeyShare_GenOtMlKemKeyClient(ssl, kse);
            break;
        default:
            ret = NOT_COMPILED_IN;
            break;
    }
    return ret;
}
```

Listing 4.2: Use `NamedGroup` enum to direct `keygen` calls to the underlying primitive

or if-else blocks, however, the explicitness also made it easy to reason about the state machine logic. We especially appreciate this flat abstraction when using a visual debugger such as `lldb`.

```rust
pub struct CryptoProvider {
    pub cipher_suites: Vec<SupportedCipherSuite>,
    pub kx_groups: Vec<&'static dyn SupportedKxGroup>,
    pub signature_verification_algorithms: WebPkiSupportedAlgorithms,
    pub secure_random: &'static dyn SecureRandom,
    pub key_provider: &'static dyn KeyProvider,
}
```

Listing 4.3: User can provide custom cryptographic backend to `rustls` through the `CryptoProvider` struct

Memory management

Programming in C brings substantial challenge in memory management, especially on the embedded client, where memory leaks can quickly exhaust the meager amount of available RAM. Following best practices, we allocate memory for cryptographic operations on the stack wherever possible, and on the heap only when necessary, such as when data need to persist across multiple TLS messages. With the ephemeral key exchange, KEM public key and private key must be dynamically allocated because they need to live after the local frame exits.

Miscellaneous

One interesting design is a `static const word16 preferredGroup[]`. User can specify the key exchange groups using the `wolfSSL_CTX_set_groups` API, but this user-specified list of groups is copmared with the internal static `preferredGroups` when constructing `ClientHello`. If the two lists do not match, then WolfSSL will send a `ClientHello` with an empty `key_share` extension; per RFC 8446, this indicates that client is requesting group selection from the server, and will increase handshake latency by one round trip. We were initially unaware of this feature and had spent a lot of effort trying to find out why the client is sending `ClientHello` twice even though client and server both support the newly added KEM.

4.1.3 Authenticate with post-quantum signatures

Because we need to distribute related components of a public-key certificate chain to the server and the client separately, we need to generate the certificate and the private key files. X.509 is the standard format for encoding a public-key certificate. It is based on the Abstract Sytnax Notation One (ASN.1) and is usually serialized according to Distinguished Encoding Rule (DER). DER is a binary format and not human-readable. We chose to further encode the DER output into PEM, which is a human-readable format that exclusively uses ASCII characters. Although the PEM-format requires a larger size to store the same certificate/private key because it is a Base64 encoding of the DER data enclosed in header and footer, it is also much easier to work with, as we could trivially load a PEM-formatted certificate into the embedded client's firmware using C macros (Listing 4.4).

WolfSSL includes an ASN.1 module that can generate, sign, encode, and parse public-key certificates and private keys (Listing 4.5).

```c
#define CA_CERT                           \
    "-----BEGIN CERTIFICATE-----"  \
    "..."                          \
    "-----END CERTIFICATE-----"
/* ... setting up WolfSSL ... */
uint8_t ca_certs[] = CA_CERT;
size_t ca_certs_size = sizeof(ca_certs);
wolfSSL_CTX_set_verify(ctx, WOLFSSL_VERIFY_PEER, NULL);
ssl_err = wolfSSL_CTX_load_verify_buffer(ctx, ca_certs, ca_certs_size,
                                          SSL_FILETYPE_PEM);
if (ssl_err != SSL_SUCCESS) {
    /* fail */
}
```

Listing 4.4: PEM-formatted certificate can be loaded without a file system

```c
int ret;
WC_RNG rng;
MlDsaKey key;
Cert cert;
uint8_t der[BUF_SIZE], pem[BUF_SIZE];
int derSz, pemSz;
/* ... initialize signature keypairs and RNG ... */
wc_InitCert(&cert);
cert.sigType = CTC_ML_DSA_LEVEL1;
/* ... fill in subject, issuer, valid dates ... */
derSz = wc_MakeCert_ex(&cert, der, sizeof(der), ML_DSA_LEVEL2_TYPE,
                       &key, &rng);
derSz = wc_SignCert_ex(cert.bodySz, cert.sigType, der, sizeof(der),
                       ML_DSA_LEVEL2_TYPE, &key, &rng);
pemSz = wc_DerToPem(der, derSz, pem, sizeof(pem), CERT_TYPE);
```

Listing 4.5: Generate, sign, then PEM-encode certificate with WolfSSL

Thanks to existing scaffolding for supporting ML-DSA, expanding WolfSSL's ASN.1 engine to support generating certificates containing ML-KEM/HQC/Falcon/SPHINCS+ public key, and to support signing certificate body with Falcon/SPHINCS+ private keys is straightforward. We particularly appreciate the clever abstraction with which WolfSSL's `MakeCert` and `SignCert` API can support more than a dozen signature scheme/variants while maintaining a compact function signature (Listing 4.6).

```
int wc_MakeCert_ex(Cert* cert, byte* derBuffer, word32 derSz,
                   int keyType, void* key, WC_RNG* rng) {
    falcon_key*        falconKey = NULL;
    dilithium_key*     dilithiumKey = NULL;
    /* ... other key types ...*/
    switch (keyType) {
        case FALCON_LEVEL1_TYPE:
        case FALCON_LEVEL5_TYPE:
            falconKey = (falcon_key *)key;
            break;
        case ML_DSA_LEVEL2_TYPE:
        case ML_DSA_LEVEL3_TYPE:
        case ML_DSA_LEVEL5_TYPE:
            dilithiumKey = (dilithium_key *)key;
            break;
        /* ... */
    }
    /* ... build cert with key and encode DER to derBuffer ... */
}
```

Listing 4.6: Using enum as hint to cast void pointers to the correct type allows `MakeCert`/`SignCert` to have compact signature despite lack of generics in C

Expanding WolfSSL's implementation to parse Falcon/SPHINCS+ public key from certificate and produce/verify Falcon/SPHINCS+ signature in `CertificateVerify` follows similar procedures, so we omit the implementation details. The one exception is the 16383-byte fragment size limit of TLS 1.3's record layer. This prevents all SPHINCS+ signatures (except for the "small" variant at NIST level 1) from being sent in a single record. While RFC 8446 allows `CertificateVerify` with signature size up to 65535 bytes to be fragmented across multiple records, unfortunately WolfSSL did not support fragmenting this message. Consequently, we were unable to test server authentication with any of SPHINCS+ fast variants nor any of the small variants at NIST level 3 or above.

4.1.4 Implementing unilaterally authenticated KEMTLS

As illustrated in Figure 2.1 and 2.4, KEMTLS and TLS 1.3 share half of the handshake workflow, and they handle exchanging application data identically. Their similarities allowed us to reuse much of WolfSSL's existing TLS 1.3 state machine, making modifications only where the handshake workflows diverge. This section describe in details these modifications.

Handling KEM certificates

We begin by adding two flags `haveMlKemAuth` and `haveHqcAuth` to the `WOLFSSL` struct. These flags indicate whether server authentication will go through KEMTLS workflow and with which scheme. If client authentication is to be implemented in future works, then we need to account for the possibility that server may wish to be authenticated with KEM while client is authenticated with signatures, hence two sets of flags are needed.

On the server side, if the leaf public key from the loaded certificate chain is a ML-KEM/HQC key, as identified by the OID, then the appropriate flag is set. After server finishes sending `Certificate`, the KEM-auth flags will decide whether the state machine will proceed to send `CertificateVerify` as in signature-based authentication, or to process `KemCiphertext` as in KEM-authentication (Listing 4.7).

On the client side, the KEM-auth flags are set during `Certificate` processing, and they decide whether the client state machine proceeds to process `CertificateVerify` or to send `KemCiphertext` (Listing 4.8).

Constructing and processing `KemCiphertext`

We follow prior implementation [17, 40] for the structure of this message: `KemCiphertext` is a handshake message (borrowing the message type `client_key_exchange` from TLS 1.2) whose body contains the raw ciphertext bytes (Listing 4.9). In our implementation, each server will load at most one private key, so the content of `KemCiphertext` will be decapsulated or rejected according to that one key. We leave for future works to design a richer structure that annotates the message with some identifier (perhaps an OID) so it can be correctly decapsulated by server loaded with multiple private keys.

```c
int wolfSSL_accept_TLSv13(WOLFSSL *ssl) {
    /*  ... earlier states ... */
    if (!ssl->options.resuming && ssl->options.sendVerify) {
        if ((ssl->error = SendTls13Certificate(ssl)) != 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    }
    if (ssl->options.haveMlKemAuth || ssl->options.haveHqcAuth) {
        ssl->options.acceptState = KEMTLS_ACCEPT_CERT_SENT;
        ssl->error = accept_KEMTLS(ssl);
        if (ssl->error == 0) {
            return WOLFSSL_SUCCESS;
        } else {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    } else {
        ssl->options.acceptState = TLS13_CERT_SENT;
    }
    if (!ssl->options.resuming && ssl->options.sendVerify) {
        if ((ssl->error = SendTls13CertificateVerify(ssl)) != 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    }
    /* ... later states ...*/
}
```

Listing 4.7: `accept_KEMTLS` will take over the handshake workflow after sending `Certificate` if either of `haveMlKemAuth` or `haveHqcAuth` is set

`Finished` and application data

`Finished` is the last handshake message in both KEMTLS and TLS 1.3. Syntactically, KEMTLS `Finished` is identical to TLS 1.3 `Finished`: an HMAC tag under the sender's `finished_key` against the running hash of the handshake transcript. Therefore we made no change to the construction or processing of this message. On the other hand, there are several semantic differences that require modification for KEMTLS. First, the pair of `finished_key` is derived with input from both the handshake secret and the authentication secret (Listing 4.10). Second, the order in which client and server's `Finished` is sent is flipped, which requires modifying the corresponding "message order sanity checks" in

```c
int wolfSSL_connect_TLSv13(WOLFSSL *ssl) {
    /* ... setup, send ClientHello, process ServerHello, etc. ... */
    while (ssl->options.serverState < SERVER_CERT_COMPLETE) {
        if ((ssl->error = ProcessReply(ssl)) < 0) {
                WOLFSSL_ERROR(ssl->error);
                return WOLFSSL_FATAL_ERROR;
        }
    }
    if (ssl->options.haveMlKemAuth || ssl->options.haveHqcAuth) {
        ssl->error = connect_KEMTLS(ssl);
        if (ssl->error != 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        } else {
            return WOLFSSL_SUCCESS;
        }
    }

    while (ssl->options.serverState < SERVER_FINISHED_COMPLETE) {
        if ((ssl->error = ProcessReply(ssl)) < 0) {
            WOLFSSL_ERROR(ssl->error);
            return WOLFSSL_FATAL_ERROR;
        }
    }
    /* ... cleanup ... */
}
```

Listing 4.8: Advance client's state machine until after `Certificate` is processed, then `connect_KEMTLS` will take over the handshake if either of `haveMlKemUath` or `haveHqcAuth` is set while processing `Certificiate`

```c
static int SendKemTlsClientKemCiphertext(WOLFSSL *ssl) {
    /* add both record and handshake headers */
    AddTls13Headers(output, ssl->kemCiphertextSz, client_key_exchange,
                    ssl);
    XMEMCPY(input + HANDSHAKE_HEADER_SZ, ssl->kemCiphertext,
            ssl->kemCiphertextSz);
    inputSz = ssl->kemCiphertextSz + HANDSHAKE_HEADER_SZ;
    sendSz = BuildTls13Message(ssl, output, outputSz, input, inputSz,
                               handshake, hashOutput, 0, 0);
    if (sendSz < 0)
        return BUILD_MSG_ERROR;
    ssl->buffers.outputBuffer.length += sendSz;
    ret = SendBuffered(ssl);
    return ret;
}
```

Listing 4.9: Client constructing `KemCiphertext`

WolfSSL.

KEMTLS's application data layer is identical to its TLS 1.3 counterpart and requires no changes, but before KEMTLS transitions to exchanging application data, we need to derive the application traffic keys and configure the handshake state machine to indicate a successful handshake. KEMTLS and TLS 1.3 both derive application traffic keys from the Master Secret (MS), so we can reuse WolfSSL's `DeriveTls13Keys` after `deriveKemTlsFinishedSecrets` derives MS from HS and $\mathtt{ss}_s$. As for configuring the handshake state machine, we observe that WolfSSL keeps track of the state of a handshake using an increasing sequence of integers (Listing 4.11). We added handshake states unique to KEMTLS while making sure that the value of individual states is consistent with the order in which a KEMTLS handshake goes through these states. To ensure that the KEMTLS handshake correctly produced a consistent pair of application traffic keys, we implemented the client to send a few random bytes after the handshake finishes, and the server to echo back client's application data. The application data do not count toward handshake performance in Section 5.4.

4.2 Embedded client networking

TLS requires a reliable transport layer such as TCP/IP. In our implementation, server-side networking is trivially handled by the Linux operating system: we only need to call

```c
static int deriveKemTlsFinishedSecrets(WOLFSSL *ssl,
                                       byte *kemSharedSecret,
                                       word32 kemShareddSecretSz) {
    /* ... after sending Finished, derive HS, dHS, AHS, dAHS, and MS */
    /* client_finished_key <- HKDF_expand(MS, "c finished", NULL) */
    DeriveKeyMsg(ssl, ssl->keys.client_write_MAC_secret, -1,
                 ssl->arrays->masterSecret, clientFinishedKeyLabel,
                 clientFinishedKeyLabelLen, NULL, 0,
                 ssl->specs.mac_algorithm);
    /* server_finished_key <- HKDF_expand(MS, "s finished", NULL) */
    DeriveKeyMsg(ssl, ssl->keys.server_write_MAC_secret, -1,
                 ssl->arrays->masterSecret, serverFinishedKeyLabel,
                 serverFinishedKeyLabelLen, NULL, 0,
                 ssl->specs.mac_algorithm);
    return 0;
}
```

Listing 4.10: Deriving `HS, dHS, AHS, dAHS, MS, finished keys`

`wolfSSL_set_fd` to connect the TLS state machine to the TCP stream. Client-side networking is non-trivial due to the lack of an operating system. Instead we had to use WolfSSL's I/O callback API and manually connect the transport layer to the TLS state machine. In this section we briefly describe how we approached implementing the transport layer on the embedded client.

We programmed our embedded client (see Section 5.1) using the Pico-SDK, which includes the `cyw43` driver for wireless connectivity and the `lwip` library for low-level protocol control. `lwip` (Lightweight IP) is an open-source TCP/IP stack developed by Adam Dunkels [30]. It is designed for minimal code size and memory usage, making it particularly well-suited for resource-constrained environments.

Unlike prior work evaluating PQ-TLS/KEMTLS [40], we chose not to use a real-time operating system (RTOS). This decision helped us avoid the added complexity of asynchronous network programming typically required with an RTOS. However, it also meant we could not use `lwip`'s higher-level socket API, which depends on RTOS features such as threading and synchronization. Instead, we used `lwip`'s raw API, which offers fine-grained control over TCP but only in an event-driven, callback-based form. Since WolfSSL's I/O callback interface is designed to work with stream-oriented transports, we needed to build a stream-like abstraction on top of `lwip`'s raw API.

`lwip`'s raw API handles data at packet level. When the network interface receives an IP packet, a handler is called to process the packet based on higher level need. To

```c
static int ssl_in_handshake(WOLFSSL *ssl, int sending_data) {
    if (ssl->options.handShakeState != HANDSHAKE_DONE)
        return 1;
    if (ssl->options.side == WOLFSSL_SERVER_END) {
        if (IsAtLeastTLSv1_3(ssl->version))
            return ssl->options.acceptState < TLS13_TICKET_SENT;
        return 0;
    }
    if (ssl->options.side == WOLFSSL_CLIENT_END) {
        if (IsAtLeastTLSv1_3(ssl->version))
            return ssl->options.connectState < FINISHED_DONE;
        return 0;
    }
    return 0;
}
```

Listing 4.11: WolfSSL checks the state of using the enum `handShakeState` and `connectState`/`acceptState`. Smaller value in this enum indicates earlier state in the handshake.

get a stream-like behavior, we implemented a buffered read: when a packet is received, the handler will append the incoming packet onto an internal buffer, then later when the application reads from the stream, a pointer keeps track of the amount of data consumed from the internal buffer. The read buffer is freed when all data have been consumed by the application to prevent memory leak. WolfSSL already buffers its write operations, so we did not implement buffered write at the stream level. Listing 4.12 shows how the buffered read is implemented.

```c
/* handle incoming packet */
static err_t tcp_recv_handler(void *arg, struct tcp_pcb *tpcb,
                              struct pbuf *p, err_t err) {
    tcp_stream_t *stream = (tcp_stream_t *)arg;
    err_t lwip_err = ERR_OK;
    if (stream == NULL) {
        /*... fail ...*/
    } else if (!p) {
        stream->terminated = true; /* peer terminated connection */
    } else if (p) {
        if (stream->rx_pbuf) {
            uint32_t remaining_cap = 0xFFFF - stream->rx_pbuf->tot_len;
            if (remaining_cap < p->tot_len) {
                lwip_err = ERR_WOULDBLOCK;
            } else {
                pbuf_cat(stream->rx_pbuf, p);
            }
        } else {
            stream->rx_pbuf = p;
        }
    }
    return lwip_err;
}

err_t tcp_stream_read(tcp_stream_t *stream, uint8_t *buf, size_t bufcap,
                      size_t *outlen) {
    /* ... */
    if (stream->rx_pbuf != NULL) {
        uint16_t remlen = stream->rx_pbuf->tot_len - stream->rx_offset;
        size_t copylen = MIN(remlen, bufcap);
        *outlen = pbuf_copy_partial(stream->rx_pbuf, buf, copylen,
                                    stream->rx_offset);
        tcp_recved(stream->pcb, *outlen);
        if (stream->rx_pbuf->tot_len == *outlen + stream->rx_offset) {
            pbuf_free(stream->rx_pbuf);
            stream->rx_pbuf = NULL;
            stream->rx_offset = 0;
        } else {
            stream->rx_offset += *outlen;
        }
    } /* ... */
}
```

Listing 4.12: Buffered read using `lwip`'s raw API

Chapter 5

Performance of post-quantum TLS 1.3 and KEMTLS

This section present and analyze performance benchmarking data. Section 5.1 describes the hardware setup and the software toolchain. Section 5.2 analyzes the communication costs, including the sizes of the various certificate chains. Section 5.3 summarizes the performance of relevant cryptographic primitives on the test hardware. Finally Section 5.4 details the performance of post-quantum TLS 1.3 and KEMTLS.

5.1 Experiment setup

Client The Raspberry Pi Pico 2 W (Figure 5.1) is a microcontroller developed by the Raspberry Pi Foundation and was released in late 2024. At the heart of the Pico 2 W is the RP2350 chip (clocked at 150MHz), which contains two ARM Cortex-M33 cores and two Hazard3 RISC-V cores, although only one architecture can be enabled at a time. The Pico 2 W also has 520KB of SRAM and 4MB of flash storage. For connectivity, the Pico 2 W has a CYW43439 module that supports 2.4 GHz Wifi and Bluetooth 5.4. Firmware is compiled using the GNU ARM toolchain (`arm-none-eabi-gcc` version 8.5.0) and `CMakeBuildType=RelWithDebInfo`.

Server The TLS server runs on a desktop computer with an 8-core AMD Ryzen 7 1700X clocked at 3.4GHz and 12GB of DDR4 RAM. The desktop runs Ubuntu 24.04 LTS.

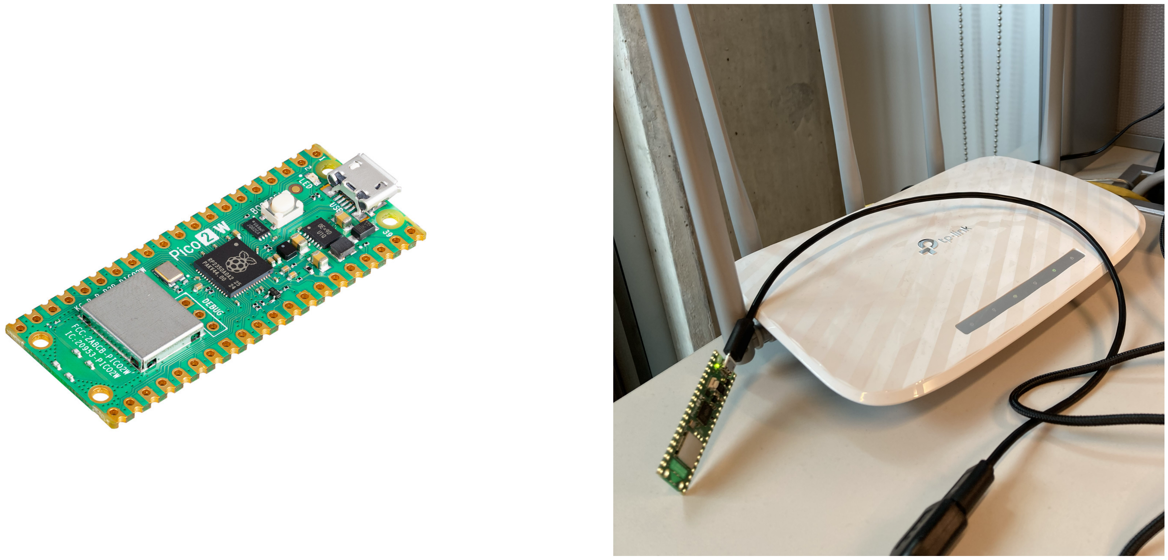

Figure 5.1: Client hardware: Pi Pico 2 W connected via 2.4GHz WiFi

The server binary is compiled using `GCC 13.3.0` and `CMAKE_BUILD_TYPE=RelWithDebInfo`. Turbo boosting was disabled from the BIOS.

Network Our network setup takes advantage of Pico's wireless capabilities. The Pico is connected to a TP-LINK wireless access point via 2.4GHz WiFi (Figure 5.1). The access point is then connected via Ethernet to the internal network of University of Waterloo. The server is directly connected to the same internal network via Ethernet, as well. According to `traceroute`, there is one more hop (an internal nameserver) between the access point and the server. The average network latency is less than 10 milliseconds. We choose to connect to an internal network with Internet access instead of a direct connection because we want to simulate a realistic peer verification policy that checks the expiration dates of peer's certificates. To achieve this, we adapted the example Network Time Protocol (NTP) client from Pico-SDK's repository [1] and synchronized time with `pool.ntp.org`. We also used `lwip`'s DNS API to resolve the server IP address from its hostname. Time synchronization and DNS lookup are both performed once at Pico 2 W's boot time and do not count toward TLS handshake measurements.

[1] https://github.com/raspberrypi/pico-examples/blob/master/pico_w/wifi/ntp_client

5.2 Communication cost

TLS 1.3 recommended ephemeral elliptic curve Diffie-Hellman key agreement (ECDHE) to be the default scheme for key exchange in `ClientHello` and `ServerHello`. Using ECDHE, the client and the server each sends the other an element from the agreed group of points on an elliptic curve, so round-trip communication cost is the sum of two group points. When using a post-quantum KEM, the client sends to the server an encapsulation key, and the server sends back a ciphertext obtained under said encapsulation key. The round-trip communication cost of a KEM key agreemnt is thus the sum of a public key and a ciphertext. Post-quantum digital signature schemes also have larger sizes than elliptic-curve signatures and RSA-signatures. The total communication cost of digital signature is the sum of public key size and signature size. Table 5.1 compares the communication costs between ECDHE and various post-quantum KEMs. Note that NIST curves (P-256, P-384, P-521) can be represented using compressed or uncompressed forms, but per RFC 8446 [86], ECDHE in TLS 1.3 always uses the uncompressed form, hence only uncompressed forms are listed.

Name	**PK**	**CT**	**RTT**
ECDHE			
P-256	65	–	130
P-384	97	–	194
P-521	133	–	266
X25519	32	–	64
X448	56	–	112
PQ KEM			
ML-KEM-512	800	768	1568
ML-KEM-768	1184	1088	2272
ML-KEM-1024	1568	1568	3136
HQC-128	2249	4433	6682
HQC-192	4522	9029	13551
HQC-256	7245	14469	21714

Scheme	**PK**	**SIG**	**Total**
Classical			
RSA-2048 (PKCS#1 v1.5)	256	256	512
ECDSA P-256	64	64	128
ECDSA P-384	96	96	192
ECDSA P-521	132	132	264
Ed25519	32	64	96
Ed448	57	114	171
Post-quantum			
ML-DSA-44	1312	2420	3732
ML-DSA-65	1952	3309	5261
ML-DSA-87	2592	4627	7219
Falcon-512	897	666	1563
Falcon-1024	1793	1280	3073
SPHINCS+-128f	32	17088	17120
SPHINCS+-192f	48	35664	35712
SPHINCS+-256f	64	49856	49920
SPHINCS+-128s	32	7856	7888
SPHINCS+-192s	48	16224	16272
SPHINCS+-256s	64	29792	29856

Table 5.1: Communication costs (bytes)

In practice, the `Certificate` message often contains a certificate chain. In our experiment setup, we generate a server certificate chain with three certificates: a leaf certificate for which the server holds the private key, an intermediate certificate authority who signs server's certificate, and a root certificate authority who signs the intermediate certificate authority's certificate. Root CA's certificate is directly compiled into the client firmware, so server only transmits the leaf certificate and the intermediate CA certificate. Table 5.2 compares the size of root CA and server's certificate chain size across a variety of configurations.

5.3 Cryptographic operations performance

We benchmarked the speed of relevant cryptographic operations on the Pico 2 W and the server. On the Pico 2 W, CPU cycle count is read from the `CYCCNT` register, located at `PPB_BASE + M33_DWT_CYCCNT_OFFSET`. On the x86 server, CPU cycle count is read from the `RDTSC` register. Results are summarized in Table 5.3 and 5.4.

5.4 Handshake latency

To analyze and compare handshake latency data, we modified the WolfSSL library to collect timestamps at important steps of the handshake on the client side. The timestamps are collected using the CPU clock, which is not a real-time clock but can precisely measure time interval (within 1-2 microseconds per 150,000 microseconds). The timestamps include:

- `ch_start`: the beginning of the handshake
- `keygen_start`: when client starts generating the ephemeral keypair
- `keygen_done`: when client finishes generating the ephemeral keypair
- `ch_sent`: when client finishes sending `ClientHello`
- `sh_start`: when client start processing `ServerHello`
- `decap_start`: when client start processing server's key share in `ServerHello`. With ECDHE, this means running the `agree` sub-routine; with KEMs, this means running the `decapsulation` sub-routine
- `decap_done`: when client finishes processing server's key share, but before updating key schedule

Root		Intermediate	Leaf	Chain
Classical				
RSA-2048	1.27	RSA-2048	RSA-2048	2.50
ECDSA (P-256)	0.73	ECDSA (P-256)	ECDSA (P-256)	1.42
ECDSA (P-384)	0.81	ECDSA (P-384)	ECDSA (P-384)	1.59
ECDSA (P-521)	0.91	ECDSA (P-521)	ECDSA (P-521)	1.79
Ed25519	0.63	Ed25519	Ed25519	1.25
Ed448	0.74	Ed448	Ed448	1.45
PQ-TLS				
Falcon-1024	4.69	Falcon-1024	ML-DSA-65	9.56
Falcon-1024	4.69	Falcon-1024	ML-DSA-87	10.42
Falcon-1024	4.68	Falcon-1024	Falcon-1024	9.34
Falcon-512	2.63	Falcon-512	ML-DSA-44	5.82
Falcon-512	2.63	ML-DSA-44	ML-DSA-44	8.77
Falcon-512	2.63	Falcon-512	Falcon-512	5.24
ML-DSA-44	5.60	ML-DSA-44	ML-DSA-44	11.17
ML-DSA-65	7.67	ML-DSA-65	ML-DSA-44	14.45
ML-DSA-65	7.67	ML-DSA-65	ML-DSA-65	15.31
ML-DSA-87	10.32	ML-DSA-87	ML-DSA-87	20.62
SPHINCS-128f	23.71	ML-DSA-44	ML-DSA-44	31.02
SPHINCS-128s	11.21	ML-DSA-44	ML-DSA-44	18.52
SPHINCS-128s	11.21	SPHINCS-128s	Falcon-512	23.57
SPHINCS-128s	11.21	SPHINCS-128s	ML-DSA-44	23.57
SPHINCS-128s	11.21	SPHINCS-128s	SPHINCS-128s	22.39
SPHINCS-192s	22.56	SPHINCS-192s	ML-DSA-65	47.69
SPHINCS-256s	40.96	SPHINCS-256s	Falcon-1024	84.23
SPHINCS-256s	40.96	SPHINCS-256s	ML-DSA-87	85.32
KEMTLS				
ML-DSA-44	5.60	ML-DSA-44	ML-KEM-512	10.48
Falcon-512	2.63	Falcon-512	ML-KEM-512	5.11
ML-DSA-44	5.60	ML-DSA-44	HQC-128	12.44
ML-DSA-65	7.67	ML-DSA-65	HQC-128	15.71
ML-DSA-65	7.67	ML-DSA-65	ML-KEM-512	13.75
ML-DSA-65	7.67	ML-DSA-65	ML-KEM-512	13.75
ML-DSA-65	7.67	ML-DSA-65	ML-KEM-768	14.27
ML-DSA-87	10.32	ML-DSA-87	ML-KEM-1024	19.23
ML-DSA-87	10.32	ML-DSA-87	ML-KEM-768	18.71
Falcon-1024	4.68	Falcon-1024	ML-KEM-1024	9.04
SPHINCS-128s	11.27	SPHINCS-128s	HQC-128	25.40
SPHINCS-128s	11.21	SPHINCS-128s	ML-KEM-512	23.44
SPHINCS-192s	22.56	SPHINCS-192s	HQC-192	51.17
SPHINCS-192s	22.56	SPHINCS-192s	ML-KEM-768	46.65
SPHINCS-256s	40.96	SPHINCS-256s	HQC-256	91.62
Falcon-1024	4.68	Falcon-1024	ML-KEM-768	8.52
SPHINCS-256s	40.96	SPHINCS-256s	ML-KEM-1024	83.94

Table 5.2: Server certificate chain size (kB).

Name	Verify
NIST level 1	
RSA-2048	2.63
Ed25519	2.52
ECDSA (P-256)	21.98
Falcon-512	0.84
ML-DSA-44	3.34
SPHINCS-128f	156.45
SPHINCS-128s	52.69
NIST level 3	
Ed448	10.93
ECDSA (P-384)	51.56
ML-DSA-65	5.72
SPHINCS-192-FAST	216.40
SPHINCS-192s	75.76
NIST level 5	
sha521ecdsa	106.35
ML-DSA-87	9.84
Falcon-1024	1.673
SPHINCS-256-FAST	230.20
SPHINCS-256s	113.61

Name	KeyGen	Agree/Encap	Decap
NIST level 1			
X25519	1.94	1.94	–
SECP256R1	32.97	32.95	–
ML-KEM-512	0.94	1.04	1.22
OT-ML-KEM-512	0.93	1.17	0.44
HQC-128	48.80	98.43	148.21
NIST level 3			
SECP384R1	79.20	79.17	–
X448	8.76	8.75	–
ML-KEM-768	1.50	1.69	1.94
OT-ML-KEM-768	1.50	1.87	0.62
HQC-192	146.05	299.63	449.69
NIST level 5			
SECP521R1	168.36	168.33	–
ML-KEM-1024	2.35	2.59	2.91
OT-ML-KEM-1024	2.35	2.86	0.83
HQC-256	272.38	547.03	821.84

Table 5.3: Signature verification and ECDHE/PQ-KEM performance on RP2350 (median million cycles/tick)

- `sh_done`: when client finishes processing `ServerHello`, which includes updating key schedule and deriving handshake secret
- `cert_start`: when client starts processing `Certificate`
- `auth_start`: when client starts the asymmetric cryptographic operation for server authentication. In signature-based workflow, this is right before client starts verifying the signature in `CertificateVerify`. In KEM-based workflow, this is right before client starts encapsulating using server's public key.
- `auth_done`: when client finishes the asymmetric cryptographic operation for server authentication
- `hs_done`: when client finishes processing server's `Finished`. Note that in signature-based workflow, this is the earliest moment when client can start sending application data, but in KEM-based workflow, client can start at an earlier moment in the handshake. Due to time constraint we did not get to implement this "early application data", but we speculate that the performance impact will be minimal.

Name	KeyGen	Sign	Verify
NIST level 1			
RSA-2048	526951	11485	165
Ed25519	125	114	354
ECDSA (P-256)	3589	3667	2474
Falcon-512	49129	14716	170
ML-DSA-44	323	1340	352
SPHINCS-128f	8642	202200	12018
SPHINCS-128s	566975	50314	4040
NIST level 3			
ECDSA (P-384)	9052	9176	6053
Ed448	488	490	1524
ML-DSA-65	589	2239	573
SPHINCS-192-FAST	13136	340445	18110
SPHINCS-192s	814181	923756	5992
NIST level 5			
ML-DSA-87	890	2354	928
sha521ecdsa	14506	14682	9548
Falcon-1024	128922	32039	347
SPHINCS-256-FAST	33788	684294	17846
SPHINCS-256s	552131	179745	9405

Name	KeyGen	Agree/Encap	Decap
NIST level 1			
x25519	332	331	–
ECDHE (P-256)	3630	3580	–
ML-KEM-512	126	148	192
OT-ML-KEM-512	128	159	70
HQC-128	7247	14857	23522
NIST level 3			
x448	1284	1288	–
ECDHE (P-384)	9154	9051	–
ML-KEM-768	209	235	293
OT-ML-KEM-768	211	250	95
HQC-192	22001	44690	68769
NIST level 5			
ECDHE (P-521)	14473	14515	–
ML-KEM-1024	322	343	418
OT-ML-KEM-1024	328	364	112
HQC-256	40459	81118	127123

Table 5.4: Signature/ECDHE/PQ-KEM performance on Ryzen 1700X (median kilocycles/tick)

Within the key exchange phase we mainly care about three metrics:

- *Total time*: the duration of the entire key exchange from the client's perspective, which is the interval from `ch_start` to `sh_done`.
- *CPU time*: the time spent constructing `ClientHello` and processing `ServerHello`, equivalently the time during which client's CPU is doing active work. It is computed as `ch_sent - ch_start + sh_done - sh_start`.
- *Crypto time*: the time spent on generating client's key share and processing server's key share. With ECDHE, this means running `keygen` and `agree` each once; with KEM, this means running `keygen` and `decap` each once. Crypto time should be a subset of CPU time, since the client needs to process other components of the two messages and update the key schedule to derive handshake secret. It is computed as `keygen_done - keygen_start + decap_done - decap_start`

For the authentication phase we mainly care about three metrics:

- *Certificate transmission time*: the duration between `sh_done` and `cert_start`. This is a heuristic of the time it took to receive server's `Certificate`, but it is not a precise measurement due to network processing happening asynchronously with client's processing `ServerHello`.
- *Crypto time*: the time spent on asymmetric cryptographic operations during the authentication phase. For signature-based authentication, this includes verifying the signature in `Certificate` verify. For KEM-based authentication, this includes encapsulating the authentication secret. Note that in our KEMTLS implementation, the authentication secret is encapsulated eagerly while processing server's `Certificate`, and constructing `KemCiphertext` only involves copying the ciphertext from the internal state to the output buffer, so authentication's crypto time does not involve `KemCiphertext`.
- *Total time*: the duration of the entire authentication phase, which counts from `cert_start` to `hs_done`.

Last but not least, we report two metrics that cover the entire handshake: *time until client application data* and *time until explicit server authentication*. When using signature-based authentication, these two numbers are identical: it is the duration from `ch_start` until the moment when client finishes sending its `Finished`. After client sends `Finished`, it can start sending application data, and the server is explicitly authenticated. When using KEM-based authentication, however, client can start sending application data after it sends `Finished` but before receiving server's `Finished`. At this moment, server is implicitly authenticated, and application data is encrypted with a key such that only the intended peer can decrypt it. Server is explicitly authenticated at the cost of one more round trip.

Table 5.5 reports the median for all statistics discussed above

5.5 Discussion

5.5.1 Post-quantum TLS performance

From Table 5.3 we can see that on the RP2350, ML-KEM-512's key generation and decapsulation is as fast if not faster than X25519's key generation and agree operation. ML-DSA-44's signature verification is slower than Ed25519 (+32% CPU cycles), while

PKC Suite			KEX			Auth		Handshake	
KEX	CA	Leaf	Crypto	CPU	Total	Cert TX	Total	App data	Exp. Auth
OT-ML-KEM-512	Falcon-512	ML-KEM-512	10.42	14.38	18.14	17.01	98.94	84.17	135.43
ML-KEM-512	Falcon-512	ML-KEM-512	15.94	19.89	23.46	17.05	98.60	89.42	140.09
ML-KEM-512	Falcon-512	Falcon-512	15.80	20.00	23.68	20.12	58.83	102.44	102.44
X25519	Ed25519	Ed25519	26.90	29.05	31.38	19.65	76.92	128.40	128.40
ML-KEM-512	Falcon-512	ML-DSA-44	15.79	20.00	25.00	21.41	82.44	128.62	128.62
ML-KEM-768	Falcon-1024	ML-KEM-768	24.24	29.48	34.31	22.80	127.49	133.16	184.63
ML-KEM-512	ML-DSA-44	ML-KEM-512	15.83	19.85	23.47	24.36	144.16	140.61	192.33
ML-KEM-1024	Falcon-1024	Falcon-1024	36.45	41.94	51.32	22.69	82.91	157.01	157.01
ML-KEM-1024	Falcon-1024	ML-KEM-1024	36.42	42.61	50.77	24.14	137.68	160.04	213.55
OT-ML-KEM-512	ML-DSA-44	ML-DSA-44	10.35	14.06	20.84	23.83	117.08	161.10	161.10
ML-KEM-512	ML-DSA-44	ML-DSA-44	15.82	19.85	22.24	25.91	123.03	171.16	171.16
ML-KEM-768	Falcon-1024	ML-DSA-65	24.28	28.99	34.30	24.87	116.52	175.06	175.06
ML-KEM-512	ML-DSA-65	ML-KEM-512	15.90	19.82	25.36	26.76	182.55	180.48	234.60
ML-KEM-768	ML-DSA-65	ML-KEM-768	24.44	29.40	34.95	31.18	187.60	200.75	252.87
ML-KEM-512	ML-DSA-65	ML-DSA-44	15.86	19.79	22.53	30.58	160.09	213.67	213.67
ML-KEM-1024	Falcon-1024	ML-DSA-87	36.45	41.96	50.87	23.63	152.64	227.28	227.28
otML-KEM-768	ML-DSA-65	ML-DSA-65	15.26	20.07	26.19	32.40	176.67	235.82	235.82
ML-KEM-768	ML-DSA-65	ML-DSA-65	24.35	29.16	33.01	32.43	176.57	242.66	242.66
ML-KEM-768	ML-DSA-87	ML-KEM-768	24.59	29.65	32.43	35.16	252.55	269.04	321.45
ML-KEM-1024	ML-DSA-87	ML-KEM-1024	36.96	42.91	52.69	36.05	265.39	300.16	359.08
otML-KEM-1024	ML-DSA-87	ML-DSA-87	22.30	27.87	36.10	37.82	272.97	347.02	347.02
ML-KEM-1024	ML-DSA-87	ML-DSA-87	36.42	42.02	51.04	38.96	274.23	364.68	364.68
ECDHE (P-256)	RSA-2048	RSA-2048	381.53	383.80	388.00	15.35	80.59	484.22	484.22
x448	Ed448	Ed448	118.48	120.70	123.23	13.83	260.90	398.13	398.13
ECDHE (P-256)	ECDSA (P-256)	ECDSA (P-256)	368.66	370.88	374.75	13.92	396.85	785.78	785.78
ML-KEM-512	SPHINCs-128s	ML-DSA-44	15.84	20.13	24.10	42.97	763.35	832.08	832.08
ML-KEM-512	SPHINCs-128s	Falcon-512	15.84	20.14	27.92	52.47	785.56	870.80	870.80
ML-KEM-512	SPHINCs-128s	ML-KEM-512	16.34	20.28	27.06	40.85	925.16	937.47	1000.18
ML-KEM-768	SPHINCs-192s	ML-KEM-768	24.25	29.16	33.59	69.06	1187.92	1237.78	1296.93
ML-KEM-768	SPHINCs-192s	ML-DSA-65	24.22	28.70	34.58	78.91	1168.51	1284.51	1284.51
ML-KEM-1024	SPHINCs-256s	Falcon-1024	36.42	41.91	51.08	116.38	1617.79	1786.86	1786.86
ML-KEM-1024	SPHINCs-256s	ML-KEM-1024	36.57	42.25	50.39	111.88	1723.76	1833.62	1888.12
ML-KEM-1024	SPHINCs-256s	ML-DSA-87	36.50	41.99	52.25	122.90	1711.70	1889.93	1889.93
ECDHE (P-384)	ECDSA (P-384)	ECDSA (P-384)	1069.65	1071.92	1080.33	16.55	1027.25	2125.43	2125.43
HQC-128	SPHINCs-128s	SPHINCs-128s	1314.94	1322.86	1340.58	50.14	1130.28	2523.21	2523.21
HQC-128	SPHINCs-128s	HQC-128	1315.06	1322.25	1339.80	49.76	1591.32	2928.46	2986.82
ECDHE (P-521)	ECDSA (P-521)	ECDSA (P-521)	2216.70	2218.99	2231.10	12.91	2082.33	4327.12	4327.12
HQC-192	SPHINCs-192s	ML-DSA-65	3993.13	4004.71	4040.27	87.24	1168.54	5297.30	5297.30
HQC-192	SPHINCs-192s	HQC-192	3993.01	4006.22	4044.01	97.01	3188.02	7278.21	7337.77
HQC-256	SPHINCs-256s	ML-DSA-87	7296.34	7314.21	7379.37	145.63	1711.86	9239.71	9239.71
HQC-256	SPHINCs-256s	HQC-256	7296.36	7315.06	7380.46	157.05	5457.84	12927.91	13003.19

Table 5.5: Median handshake statistics (ms)

Falcon-512's signature verification is faster (-67% CPU cycles). Overall, a post-quantum TLS handshake utilizing lattice-based KEM and signature has competitive latency with handshakes using elliptic-curve key exchange and signatures. In production deployment with fast servers and broadband networks, lattice-based primitives' slower signing routine and larger message sizes do not incur noticeable performance penalty; instead, a TLS handshake is bottlenecked by the embedded client's cryptographic operations. Table 5.6 and Figure 5.2 the performance of lattice-based handshake and elliptic-curve handshake.

PKC suite		**KEX**	**Certs TX**	**Auth**	**Handshake**
KEX	Auth				
X25519	Ed25519	31.38 ms	19.65 ms	76.92 ms	128.40 ms
ECDHE (P-256)	RSA-2048	388.00 ms	15.35 ms	80.59 ms	484.22 ms
ECDHE (P-256)	ECDSA (P-256)	374.75 ms	13.92 ms	396.85 ms	785.78 ms
ML-KEM-512	ML-DSA-44	22.24 ms	25.91 ms	123.03 ms	171.16 ms
ML-KEM-512	Falcon-512	23.68 ms	20.12 ms	58.83 ms	102.44 ms
X448	Ed448	123.23 ms	13.83 ms	260.90 ms	398.13 ms
ECDHE (P-384)	ECDSA (P521)	1080.33 ms	16.55 ms	1027.25 ms	2125.43 ms
ML-KEM-768	ML-DSA-65	33.01 ms	32.43 ms	176.57 ms	242.66 ms
ECDHE (P-521)	ECDSA (P521)	2231.10 ms	12.91 ms	2082.33 ms	4327.12 ms
ML-KEM-1024	ML-DSA-87	51.04 ms	38.96 ms	274.23 ms	364.68 ms
ML-KEM-1024	Falcon-1024	51.32 ms	22.69 ms	82.91 ms	157.01 ms

Table 5.6: lattice-based vs elliptic-curve TLS handshake

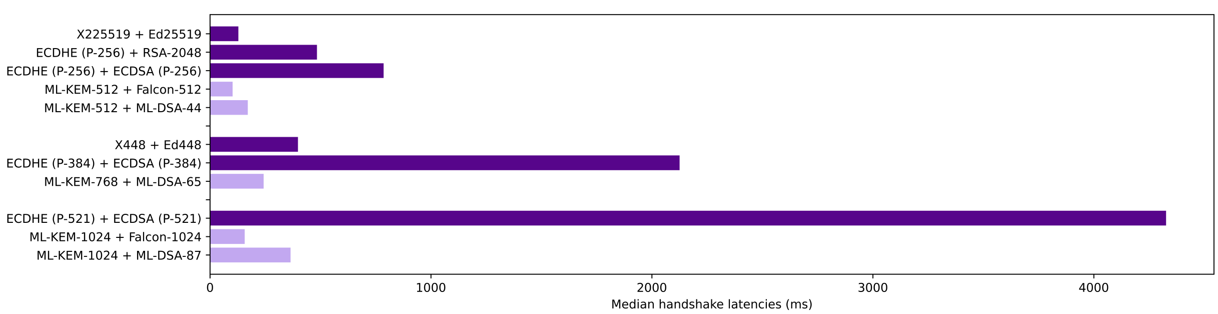

Figure 5.2: elliptic-curve vs lattice in TLS handshake

Falcon has generally the best overall performance amongst all post-quantum signature schemes due to its compact public key and signatures, and its fast verification on the client. However, Falcon's signing routine needs to sample from a discrete Gaussian distribution [37], which requires *constant-time floating-point arithmetic with at least 53-bit precision*

[33]. This requirement can be hard to fulfill, especially on legacy hardware, so it might not be feasible to use Falcon for online signatures. Fortunately, we can still use Falcon for offline signatures and still see meaningful performance improvements. Table 5.7 and Figure 5.3 compare using Falcon as online signatures and using Falcon as offline signatures.

PKC suite			Certs TX	Auth	Handshake
KEX	CA	Leaf			
ML-KEM-512	Falcon-512	Falcon-512	20.12 ms	58.83 ms	102.44 ms
	Falcon-512	ML-DSA-44	21.41 ms	82.44 ms	128.62 ms
	ML-DSA-44	ML-DSA-44	25.91 ms	123.03 ms	171.16 ms
ML-KEM-768	ML-DSA-65	ML-DSA-65	32.43 ms	176.57 ms	242.66 ms
	Falcon-1024	ML-DSA-65	24.87 ms	116.52 ms	175.06 ms
ML-KEM-1024	Falcon-1024	Falcon-1024	22.69 ms	82.91 ms	157.01 ms
	Falcon-1024	ML-DSA-87	23.63 ms	152.64 ms	227.28 ms
	ML-DSA-87	ML-DSA-87	38.96 ms	274.23 ms	364.68 ms

Table 5.7: Comparing Falcon online, Falcon offline, and ML-DSA

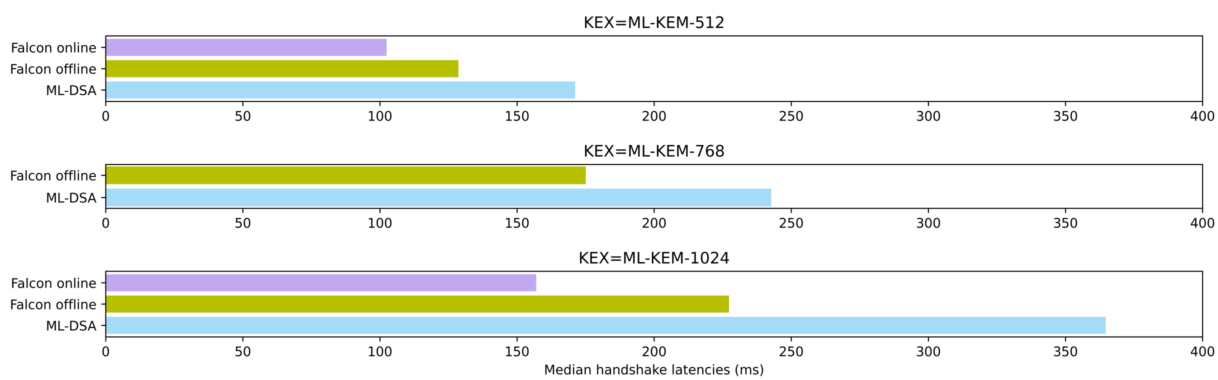

Figure 5.3: Comparing Falcon online, Falcon offline, and ML-DSA

In real-world deployments, Certificate Authorities' public keys usually have much longer lifetime than leaf public keys. For example, in the certificate chain for `www.github.com`, the leaf certificate is valid for 1 year, and intermediate CA is valid for 12 years, and the root CA is valid for 28 years. Therefore, CA's may require using signature schemes at a higher security level or with a more conservative security assumption. Table 5.8 and Figure 5.4 summarize the performance of using a mixture of security level for CA certificates.

Using HQC for key exchange (or KEMTLS) is currently not advisable. Not only is HQC significantly slower than ML-KEM for key exchange or authentication, the most recent

	PKC suite		**Certs TX**	**Auth**	**Handshake**
KEX	CA	Leaf			
ML-KEM-512	ML-DSA-44	ML-DSA-44	25.91 ms	123.03 ms	171.16 ms
	ML-DSA-65	ML-DSA-44	30.58 ms	160.09 ms	213.67 ms
	SPHINCS-128s	ML-DSA-44	42.97 ms	763.35 ms	832.08 ms
ML-KEM-768	ML-DSA-65	ML-DSA-65	32.43 ms	176.57 ms	242.66 ms
	Falcon-1024	ML-DSA-65	24.87 ms	116.52 ms	175.06 ms
	ML-DSA-87	ML-DSA-65	37.09 ms	245.43 ms	315.08 ms
	SPHINCS-192s	ML-DSA-65	78.91 ms	1168.51 ms	1284.51 ms
ML-KEM-1024	ML-DSA-87	ML-DSA-87	38.96 ms	274.23 ms	364.68 ms
	SPHINCS-256s	ML-DSA-87	122.90 ms	1711.70 ms	1889.93ms

Table 5.8: Certificate Authority with upgraded security levels

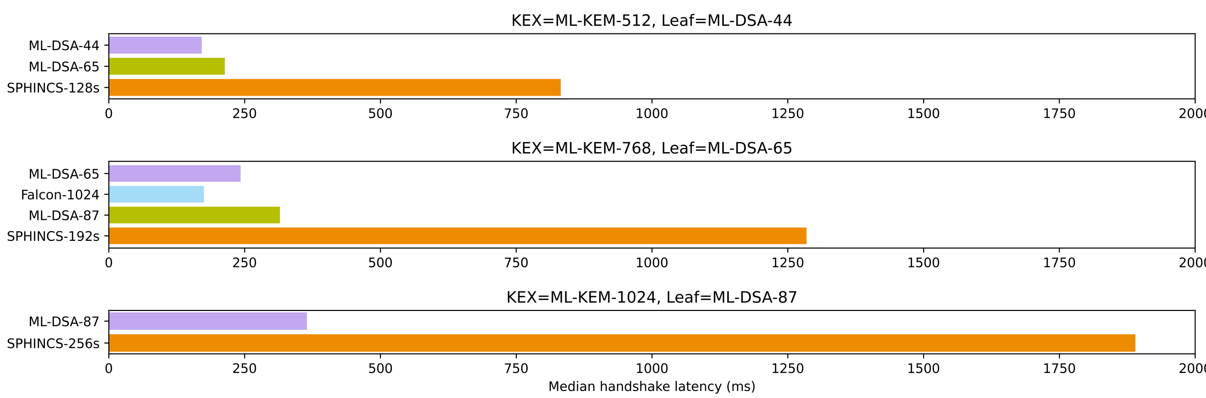

Figure 5.4: Certificate Authority with upgraded security levels

reference implementation (April 2025) was also found to have a flawed FO-transform that damages the CCA security [2]. At the time of writing this thesis, the CCA security flaw in HQC's reference implementation is not yet resolved.

5.5.2 IND-1CCA KEM vs IND-CCA KEM

A significant percentage of key exchange time is spent on cryptographic operations from the client side, and the slower the KEM, the more KEM operations dominate the performance of key exchange. As expected, the decapsulation savings of IND-1CCA KEM brought significant speedup of the key exchange phase. However, improvements in key exchange time is drowned out by the far more time consuming authentication phase, switching to IND-1CCA KEM reduced handshake latency by only up to 5%. Table 5.9 and Figure 5.5 compare the performance of handshake between these two classes of KEMs for key exchange.

KEM	Auth	KEX time	Auth time	Handshake time	
				This work	Prior work [110]
ML-KEM-512	ML-DSA-44	22.24 ms	123.03 ms	171.16 ms	3416.02 conn/sec
OT-ML-KEM-512	ML-DSA-44	20.84 ms	117.08 ms	161.10 ms	3476.17 conn/sec
ML-KEM-768	ML-DSA-65	33.01 ms	176.58 ms	242.67 ms	2710.85 conn/sec
OT-ML-KEM-512	ML-DSA-44	26.19 ms	176.68 ms	235.83 ms	2803.24 conn/sec
ML-KEM-1024	ML-DSA-87	51.04 ms	274.23 ms	364.68 ms	–
OT-ML-KEM-1024	ML-DSA-87	36.10 ms	272.97 ms	347.02 ms	–

Table 5.9: Comparing ML-KEM and OT-ML-KEM-512

5.5.3 KEM-based vs. signature-based authentication

KEM-based authentication has two main performance advantage over signature-based authentication. First, at comparable security level, ML-KEM public keys are smaller than Falcon/ML-DSA public keys, and ML-KEM ciphertexts are smaller than ML-DSA ciphertexts. Smaller public keys means a smaller server certificate chain and thus less time spent on transmitting the certificate. In practice, the smaller certificate chain indeed correlates with a shorter certficate transmission time from the client's perspective (a.k.a. time between `sh_done` and `cert_start`). However, the time savings here are minimal and easily erased with network noise in real-world deployment.

[2] https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/Wiu4ZQo3fP8

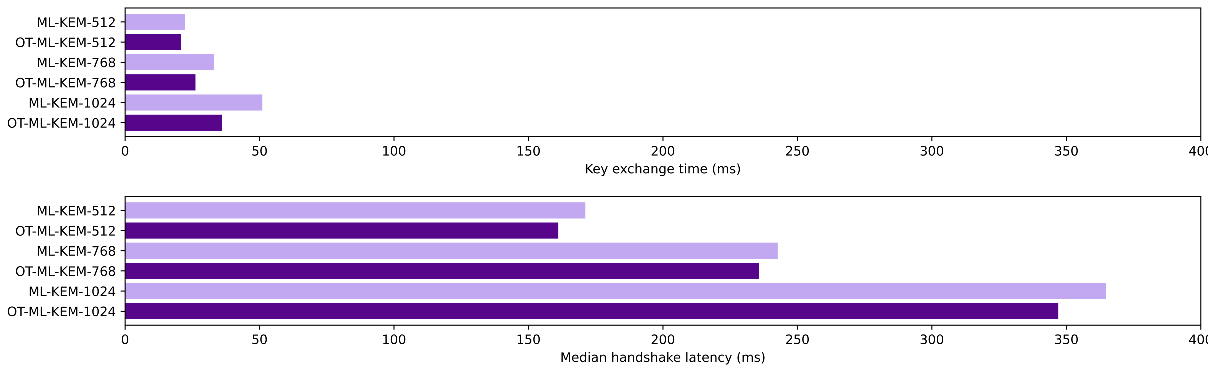

Figure 5.5: Comparing ML-KEM and OT-ML-KEM

Second, KEM-based authentication allows client to send implicitly authenticated application data before receiving server's `Finished`, whereas signature-based authentication requires client to send application data only after explicit server authentication. In signature-based authentication, client needs to wait for server's public-key cryptographic operation (signing the transcript in `CertificateVerify`), but in KEM-based authentication, client can send application data before server's public-key cryptographic operation (decapsulating `KemCiphertext`). As pointed out in [90], sending implicitly authenticated application data is slightly less secure than explicit authentication, but the security of implicitly authenticated data will be retroactively upgraded to explicit authentication after processing server's `Finished`, and there is no practical attack exploiting the gap between implicit and explicit authentication.

To summarize, compared to signature-based authentication, KEM-based authentication provides small savings public-key certificate size and moderate reduction in latency until client can start sending application data. A comparison can be found in Table 5.10 and Figure 5.6.

CA	Leaf	Certificates	Cert TX	App data	Exp. auth
Falcon-512	ML-KEM-512	5.11 kB	17.05 ms	89.42 ms	140.09 ms
	Falcon-512	5.24 kB	20.12 ms	102.44 ms	102.44 ms
	ML-DSA-44	5.81 kB	21.41 ms	171.16 ms	171.16 ms
ML-DSA-44	ML-KEM-512	10.47 kB	24.36 ms	140.61 ms	192.33 ms
	ML-DSA-44	11.17 kB	25.99 ms	161.10 ms	161.10 ms
ML-DSA-65	ML-KEM-512	13.75 kB	26.76 ms	180.48 ms	234.60 ms
	ML-DSA-44	14.44 kB	30.58 ms	213.67 ms	213.67 ms
	ML-KEM-768	14.27 kB	31.18 ms	200.75 ms	252.87 ms
	ML-DSA-65	15.31 kB	32.43 ms	235.82 ms	235.82 ms
Falcon-1024	ML-KEM-1024	9.04 kB	24.14 ms	160.04 ms	213.55 ms
	Falcon-1024	9.33 kB	22.69 ms	157.01 ms	157.01 ms
	ML-DSA-87	10.42 kB	23.63 ms	227.28 ms	227.28 ms
ML-DSA-87	ML-KEM-1024	19.22 kB	36.05 ms	300.16 ms	359.08 ms
	ML-DSA-87	20.61 kB	38.34 ms	364.68 ms	364.68 ms

Table 5.10: KEM-based vs. signature-based authentication

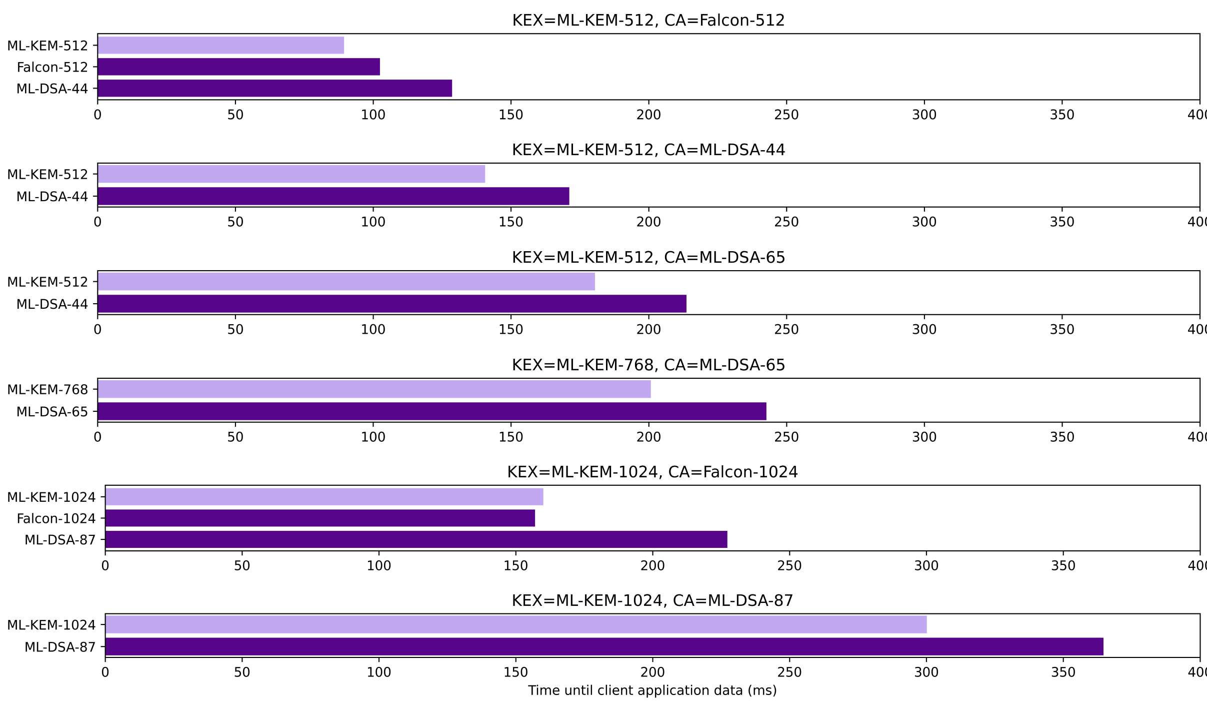

Figure 5.6: KEM-based vs. signature-based authentication

Chapter 6

Conclusion and future works

6.1 Summary of thesis

In this thesis, we proposed an original IND-1CCA KEM transformation and applied it to post-quantum TLS handshake on an embedded device, achieving meaningful performance improvements.

In Chapter 2, we reviewed the handshake protocol of TLS 1.3 and KEMTLS. Both protocols used ephemeral key exchange to encrypt subsequent traffic, but they took different approaches to authenticate peers: TLS 1.3 authenticates peer using digital signatures, and KEMTLS authenticates peer using CCA-secure KEMs. Both protocols allow clients to send application data after 1-RTT, but the different authentication approaches brought subtle security implications: in signature-based authentication, 1-RTT application data is explicitly authenticated, but in KEM-based authentication, 1-RTT application data is only implicitly authenticated. KEMTLS can still achieve explicitly authenticated application data, but only at the cost of one more round trip.

In Chapter 3 we introduced "bounded CCA security (IND-1CCA)", then proposed a generic IND-1CCA KEM transformation that builds from a passively secure PKE, wihch we called the "encrypt-then-MAC" transformation. When applied to the CPA subroutines of ML-KEM, the resulting KEM (which we call one-time ML-KEM) reduced decapsulation CPU cycles by more than 70%, while incurring less than 10% penalty in encapsulation. Our KEM transformation (and other 1CCA secure KEMs) is suitable for ephemeral key exchange in TLS 1.3 and KEMTLS.

In Chapter 4 we discussed how we generated post-quantum certificate chains and implemented PQ-TLS/KEMTLS client/server, including a baremetal TCP client stack. Our implementation is the first to converge all dependencies onto a single open-source TLS library (WolfSSL), so future works evaluating PQ-TLS/KEMTLS can have an easier time managing different external libraries.

Finally, Chapter 5 presents detailed latency metrics for PQ-TLS/KEMTLS handshake between an embedded client and a desktop server. The data not only proved feasibility of both protocols at all NIST security levels, we also drew some practical recommendations for real-world deloyments, such as using 1CCA KEM for ephemeral key exchange and prefer Falcon or ML-DSA for offline signatures.

6.2 Open problems and future works

6.2.1 Authenticating embedded device

We did not test mutual authentication on the embedded client, nor did we test the embedded device as a TLS server, because both scenarios require authenticating an embedded device. The main concern with authenticating an embedded device is the safety of the secret key. Embedded devices often operate in hostile environments where the risks of physical instrusion and sophisticated fault attacks are non-trival. While many platforms include some "secure boot" features that can increase the cost of dumping firmware and extracting secrets, they are not impossible to defeat.

On the other hand, if there is a way to safely store secrets (e.g. with a TPM [84]), then the embedded device can use TLS in PSK mode, which is far more efficient because authentication is done with symmetric primitives instead of public-key cryptography. In summary, the scenario in which authenticating an embedded device with public-key primitives remains to be found.

6.2.2 Caching peer's public key

In this thesis, we adopted the most commonly deployed public-key infrastructure architecture, where a Certificate Authority serves as a root of trust from which a chain of signatures and public key estalishes the identity of intermediate certificate authorities and ultimately the end entity. This "chain-of-trust" architecture works well when web clients need to authenticate a large number of previously unknown web servers, but it is inefficient for an IoT device that might only talk to one or two servers throughout its lifetime. Instead, for clients that talk to a fixed and known list of peers, it is more efficient to directly embed peers' public keys instead of going through intermediate and root CAs. In fact, RFC 7924 [88] already proposed a `cache_information` extension to `ClientHello` in TLS 1.2, which would allow the client to send a hash of server's public key and indicate possession, and the server can then skip sending `Certificate` and save a substantial amount of bandwidth.

Interestingly, caching server's public key would also enable the use of KEM and digital signature schemes that are previously impractical. A prominent example is Classic McEliece: even with the smallest parameter set, Classic McEliece's public key size is 250kB, which is impossible to fit into `ClientHello` even with record fragmentation (there was an IETF proposal[1] to modify `key_share` extension so Classic McEliece public keys can fit, but it required extensive work and could break backwards compatibility with TLS middle boxes, so progress was stalled). On the other hand, Classic McEliece boasts impressively small ciphertext and fast encryption, which makes it a great choice for static key exchange. We speculate that the combination of IND-1CCA KEM for ephemeral key exchange and Classic McEliece for KEM-based authentication can greatly reduce the handshake latency and computational load of a TLS handshake on an embedded client. The same argument applies to digital signatures that have large public keys and small signatures, such as Unbalanced Oil and Vinegar (UOV), a multivariate signature scheme. With the smallest parameter set, UOV's public key size is 43576 bytes and its signature size is 128 bytes. Table 6.1 summarizes the other parameter sizes.

Name	**NIST Sec. Lvl**	**PubKey**	**Ciphertext/Sig**
mceliece348864	1	261.12 kB	96 B
mceliece460896	3	524.16 kB	156 B
mceliece6688128	5	1044.99 kB	208 B
mceliece6960119	5	1047.32 kB	194 B
mceliece8192128	5	1357.82 kB	208 B
UOV-Ip	1	43.58 kB	128 B
UOV-Is	1	66.58 kB	96 B
UOV-III	3	189.23 kB	200 B
UOV-V	5	446.99 kB	260 B

Table 6.1: Post-quantum KEM/Sig schemes with large keys but small ciphertext/signatures are great candidates for public key caching

[1] https://www.ietf.org/archive/id/draft-wagner-tls-keysharepqc-00.html

References

[1] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange - A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016*, pages 327–343. USENIX Association, 2016.

[2] Manuel Barbosa, Deirdre Connolly, João Diogo Duarte, Aaron Kaiser, Peter Schwabe, Karoline Varner, and Bas Westerbaan. X-wing: The hybrid KEM you've been looking for. *IACR Cryptol. ePrint Arch.*, page 39, 2024.

[3] Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO '93, 13th Annual International Cryptology Conference, Santa Barbara, California, USA, August 22-26, 1993, Proceedings*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249. Springer, 1993.

[4] Mihir Bellare and Phillip Rogaway. Optimal asymmetric encryption. In Alfredo De Santis, editor, *Advances in Cryptology - EUROCRYPT '94, Workshop on the Theory and Application of Cryptographic Techniques, Perugia, Italy, May 9-12, 1994, Proceedings*, volume 950 of *Lecture Notes in Computer Science*, pages 92–111. Springer, 1994.

[5] Daniel J. Bernstein. The Poly1305-AES Message-Authentication Code. In Henri Gilbert and Helena Handschuh, editors, *Fast Software Encryption: 12th International Workshop, FSE 2005, Paris, France, February 21-23, 2005, Revised Selected Papers*, volume 3557 of *Lecture Notes in Computer Science*, pages 32–49. Springer, 2005.

[6] Daniel J. Bernstein. FO derandomization sometimes damages security. Cryptology ePrint Archive, Paper 2021/912, 2021.

[7] Daniel J. Bernstein, Tung Chou, Carlos Cid, Joachim Gilcher, Tanja Lange, Vamsi Maram, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Nicolas Sendrier, Jakub Szefer, Clelia Jane Tjhai, Martin Tomlinson, and Wen Wang. Classic McEliece Algorithm Specifications and Supporting Documentation. Technical report, National Institute of Standards and Technology, 2022. 4th Round submission to the NIST's Post-Quantum Cryptography Standardization Process.

[8] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS+ signature framework. *IACR Cryptol. ePrint Arch.*, page 1086, 2019.

[9] John Black and Phillip Rogaway. CBC MACs for arbitrary-length messages: The three-key constructions. *J. Cryptol.*, 18(2):111–131, 2005.

[10] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018*, pages 353–367. IEEE, 2018.

[11] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. *SIGACT News*, 28(2):14–19, 1997.

[12] Jacqueline Brendel, Marc Fischlin, Felix Günther, and Christian Janson. PRF-ODH: relations, instantiations, and impossibility results. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology - CRYPTO 2017 - 37th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 20-24, 2017, Proceedings, Part III*, volume 10403 of *Lecture Notes in Computer Science*, pages 651–681. Springer, 2017.

[13] Christina Brzuska, Marc Fischlin, Bogdan Warinschi, and Stephen C. Williams. Composability of bellare-rogaway key exchange protocols. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *Proceedings of the 18th ACM Conference on Computer and Communications Security, CCS 2011, Chicago, Illinois, USA, October 17-21, 2011*, pages 51–62. ACM, 2011.

[14] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems. *IACR Cryptol. ePrint Arch.*, page 308, 2020.

[15] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems: Integrating and evaluating kyber and SPHINCS+ with mbed TLS. In Hung-Min Sun, Shiuh-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese, editors, *ASIA CCS '20: The 15th ACM Asia Conference on Computer and Communications Security, Taipei, Taiwan, October 5-9, 2020*, pages 841–852. ACM, 2020.

[16] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: an efficient post-quantum commutative group action. *IACR Cryptol. ePrint Arch.*, page 383, 2018.

[17] Sofía Celi, Armando Faz-Hernández, Nick Sullivan, Goutam Tamvada, Luke Valenta, Thom Wiggers, Bas Westerbaan, and Christopher A. Wood. Implementing and measuring KEMTLS. In Patrick Longa and Carla Ràfols, editors, *Progress in Cryptology - LATINCRYPT 2021 - 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 6-8, 2021, Proceedings*, volume 12912 of *Lecture Notes in Computer Science*, pages 88–107. Springer, 2021.

[18] Sofía Celi, Jonathan Hoyland, Douglas Stebila, and Thom Wiggers. A tale of two models: Formal verification of KEMTLS via tamarin. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part III*, volume 13556 of *Lecture Notes in Computer Science*, pages 63–83. Springer, 2022.

[19] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 1825–1842. ACM, 2017.

[20] Ronald Cramer, Goichiro Hanaoka, Dennis Hofheinz, Hideki Imai, Eike Kiltz, Rafael Pass, Abhi Shelat, and Vinod Vaikuntanathan. Bounded CCA2-secure encryption. In Kaoru Kurosawa, editor, *Advances in Cryptology - ASIACRYPT 2007, 13th International Conference on the Theory and Application of Cryptology and Information Security, Kuching, Malaysia, December 2-6, 2007, Proceedings*, volume 4833 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2007.

[21] Eric Crockett, Christian Paquin, and Douglas Stebila. Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. *IACR Cryptol. ePrint Arch.*, page 858, 2019.

[22] Alexander W. Dent. A designer's guide to KEMs. In Kenneth G. Paterson, editor, *Cryptography and Coding, 9th IMA International Conference, Cirencester, UK, December 16-18, 2003, Proceedings*, volume 2898 of *Lecture Notes in Computer Science*, pages 133–151. Springer, 2003.

[23] Tim Dierks and Eric Rescorla. The transport layer security (TLS) protocol version 1.2. *RFC*, 5246:1–104, 2008.

[24] Jintai Ding, Ming-Shing Chen, Albrecht Petzoldt, Dieter Schmidt, and Bo-Yin Yang. Rainbow. Technical report, National Institute of Standards and Technology, 2019. Available at https://csrc.nist.gov/projects/post-quantum-cryptography/round-2-submissions.

[25] Ronny Döring and Marc Geitz. Post-quantum cryptography in use: Empirical analysis of the TLS handshake performance. In *2022 IEEE/IFIP Network Operations and Management Symposium, NOMS 2022, Budapest, Hungary, April 25-29, 2022*, pages 1–5. IEEE, 2022.

[26] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol candidates. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 1197–1210. ACM, 2015.

[27] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 draft-10 full and pre-shared key handshake protocol. *IACR Cryptol. ePrint Arch.*, page 81, 2016.

[28] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol. *J. Cryptol.*, 34(4):37, 2021.

[29] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):238–268, 2018.

[30] Adam Dunkels. Design and implementation of the lwip tcp/ip stack. *Swedish Institute of Computer Science*, 2, 03 2001.

[31] Cynthia Dwork, Moni Naor, and Omer Reingold. Immunizing encryption schemes from decryption errors. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings*, volume 3027 of *Lecture Notes in Computer Science*, pages 342–360. Springer, 2004.

[32] Marc Fischlin and Felix Günther. Multi-stage key exchange and the case of Google's QUIC protocol. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014*, pages 1193–1204. ACM, 2014.

[33] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON: Fast-Fourier Lattice-based Compact Signatures over NTRU. Technical Report Specification v1.2, National Institute of Standards and Technology, October 2020.

[34] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554. Springer, 1999.

[35] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *J. Cryptol.*, 26(1):80–101, 2013.

[36] Eiichiro Fujisaki, Tatsuaki Okamoto, David Pointcheval, and Jacques Stern. RSA-OAEP is secure under the RSA assumption. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, volume 2139 of *Lecture Notes in Computer Science*, pages 260–274. Springer, 2001.

[37] Xinwei Gao, Lin Li, Jintai Ding, Jiqiang Liu, R. V. Saraswathy, and Zhe Liu. Fast discretized gaussian sampling and post-quantum TLS ciphersuite. In Joseph K. Liu and Pierangela Samarati, editors, *Information Security Practice and Experience - 13th International Conference, ISPEC 2017, Melbourne, VIC, Australia, December 13-15, 2017, Proceedings*, volume 10701 of *Lecture Notes in Computer Science*, pages 551–565. Springer, 2017.

[38] Carlos Rubio Garcia, Abraham Cano Aguilera, Juan Jose Vegas Olmos, Idelfonso Tafur Monroy, and Simon Rommel. Quantum-resistant TLS 1.3: A hybrid solution combining classical, quantum and post-quantum cryptography. In *28th IEEE International Workshop on Computer Aided Modeling and Design of Communication Links and Networks , CAMAD 2023, Edinburgh, United Kingdom, November 6-8, 2023*, pages 246–251. IEEE, 2023.

[39] Shafi Goldwasser and Silvio Micali. Probabilistic encryption and how to play mental poker keeping secret all partial information. In Harry R. Lewis, Barbara B. Simons, Walter A. Burkhard, and Lawrence H. Landweber, editors, *Proceedings of the 14th Annual ACM Symposium on Theory of Computing, May 5-7, 1982, San Francisco, California, USA*, pages 365–377. ACM, 1982.

[40] Ruben Gonzalez and Thom Wiggers. KEMTLS vs. post-quantum TLS: performance on embedded systems. In Lejla Batina, Stjepan Picek, and Mainack Mondal, editors, *Security, Privacy, and Applied Cryptography Engineering - 12th International Conference, SPACE 2022, Jaipur, India, December 9-12, 2022, Proceedings*, volume 13783 of *Lecture Notes in Computer Science*, pages 99–117. Springer, 2022.

[41] Joint Task Force Transformation Initiative Interagency Working Group. Security and privacy controls for federal information systems and organizations. Technical Report NIST Special Publication (SP) 800-53, Rev. 4, Includes updates as of January 22, 2015, National Institute of Standards and Technology, Gaithersburg, MD, 2013.

[42] Lov K. Grover. A fast quantum mechanical algorithm for database search. In Gary L. Miller, editor, *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 212–219. ACM, 1996.

[43] Felix Günther. *Modeling Advanced Security Aspects of Key Exchange and Secure Channel Protocols*. PhD thesis, Darmstadt University of Technology, Germany, 2018.

[44] Felix Günther, Simon Rastikian, Patrick Towa, and Thom Wiggers. KEMTLS with delayed forward identity protection in (almost) a single round trip. In Giuseppe Ateniese and Daniele Venturi, editors, *Applied Cryptography and Network Security - 20th International Conference, ACNS 2022, Rome, Italy, June 20-23, 2022, Proceedings*, volume 13269 of *Lecture Notes in Computer Science*, pages 253–272. Springer, 2022.

[45] Yacoub Hanna, Diana Pineda, Maryna Veksler, Manish Paudel, Kemal Akkaya, Mila Anastasova, and Reza Azarderakhsh. Integrating post-quantum TLS into the control

plane of 5g networks. In *IEEE International Performance, Computing, and Communications Conference, IPCCC 2024, Orlando, FL, USA, November 22-24, 2024*, pages 1–8. IEEE, 2024.

[46] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In Joe Buhler, editor, *Algorithmic Number Theory, Third International Symposium, ANTS-III, Portland, Oregon, USA, June 21-25, 1998, Proceedings*, volume 1423 of *Lecture Notes in Computer Science*, pages 267–288. Springer, 1998.

[47] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371. Springer, 2017.

[48] Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Failing gracefully: Decryption failures and the Fujisaki-Okamoto transform. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part IV*, volume 13794 of *Lecture Notes in Computer Science*, pages 414–443. Springer, 2022.

[49] Loïs Huguenin-Dumittan and Serge Vaudenay. On IND-qCCA security in the ROM and its applications - CPA security is sufficient for TLS 1.3. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 613–642. Springer, 2022.

[50] Andreas Hülsing, Jaap Rijneveld, Jean-Marie Schanck, and Peter Schwabe. NTRU-HRSS-KEM: Algorithm Specifications and Supporting Documentation. Technical report, National Institute of Standards and Technology, 2017.

[51] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *Proceedings of the 39th Annual ACM Symposium on Theory of Computing, San Diego, California, USA, June 11-13, 2007*, pages 21–30. ACM, 2007.

[52] Tetsu Iwata and Kaoru Kurosawa. OMAC: one-key CBC MAC. In Thomas Johansson, editor, *Fast Software Encryption, 10th International Workshop, FSE 2003, Lund, Sweden, February 24-26, 2003, Revised Papers*, volume 2887 of *Lecture Notes in Computer Science*, pages 129–153. Springer, 2003.

[53] David Jao and Luca De Feo. Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29 - December 2, 2011. Proceedings*, volume 7071 of *Lecture Notes in Computer Science*, pages 19–34. Springer, 2011.

[54] Panos Kampanakis and Will Childs-Klein. The impact of data-heavy, post-quantum TLS 1.3 on the time-to-last-byte of real-world connections. *IACR Cryptol. ePrint Arch.*, page 176, 2024.

[55] Panos Kampanakis and Michael G. Kallitsis. Faster post-quantum TLS handshakes without intermediate CA certificates. In Shlomi Dolev, Jonathan Katz, and Amnon Meisels, editors, *Cyber Security, Cryptology, and Machine Learning - 6th International Symposium, CSCML 2022, Be'er Sheva, Israel, June 30 - July 1, 2022, Proceedings*, volume 13301 of *Lecture Notes in Computer Science*, pages 337–355. Springer, 2022.

[56] Panos Kampanakis and Dimitrios Sikeridis. Two PQ signature use-cases: Non-issues, challenges and potential solutions. Cryptology ePrint Archive, Paper 2019/1276, 2019.

[57] Matthias J. Kannwischer, Markus Krausz, Richard Petri, and Shang-Yi Yang. pqm4: Benchmarking NIST additional post-quantum signature schemes on microcontrollers. *IACR Cryptol. ePrint Arch.*, page 112, 2024.

[58] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, page 844, 2019.

[59] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving software quality in cryptography standardization projects. In *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, pages 19–30. IEEE, 2022.

[60] Aviad Kipnis, Jacques Patarin, and Louis Goubin. Unbalanced oil and vinegar signature schemes. In Jacques Stern, editor, *Advances in Cryptology - EUROCRYPT*

'99, *International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding*, volume 1592 of *Lecture Notes in Computer Science*, pages 206–222. Springer, 1999.

[61] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*, volume 6223 of *Lecture Notes in Computer Science*, pages 631–648. Springer, 2010.

[62] Hugo Krawczyk and Pasi Eronen. Hmac-based extract-and-expand key derivation function (HKDF). *RFC*, 5869:1–14, 2010.

[63] Hugo Krawczyk and Hoeteck Wee. The OPTLS protocol and TLS 1.3. In *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*, pages 81–96. IEEE, 2016.

[64] L LAMPORT. Constructing digital signatures from a one-way function. *Report SRI Intl. CSL 98*, 1979.

[65] Adam Langley. Cecpq1. https://www.imperialviolet.org/2016/11/28/cecpq1.html, 2016. Accessed: 2025-07-13.

[66] Adam Langley. Cecpq2. https://www.imperialviolet.org/2018/12/12/cecpq2.html, 2018. Accessed: 2025-07-13.

[67] Dimitri Mankowski, Thom Wiggers, and Veelasha Moonsamy. TLS to post-quantum TLS: inspecting the TLS landscape for PQC adoption on android. *IACR Cryptol. ePrint Arch.*, page 734, 2023.

[68] Dominik Marchsreiter and Johanna Sepúlveda. Hybrid post-quantum enhanced TLS 1.3 on embedded devices. In *25th Euromicro Conference on Digital System Design, DSD 2022, Maspalomas, Spain, August 31 - Sept. 2, 2022*, pages 905–912. IEEE, 2022.

[69] Robert J McEliece. A public-key cryptosystem based on algebraic coding theory. *Coding Thv*, 4244(1978):114–116, 1978.

[70] David A. McGrew and John Viega. The security and performance of the galois/-counter mode (GCM) of operation. In Anne Canteaut and Kapalee Viswanathan, editors, *Progress in Cryptology - INDOCRYPT 2004, 5th International Conference on Cryptology in India, Chennai, India, December 20-22, 2004, Proceedings*, volume 3348 of *Lecture Notes in Computer Science*, pages 343–355. Springer, 2004.

[71] Callum McLoughlin, Clémentine Gritti, and Juliet Samandari. Full post-quantum datagram TLS handshake in the internet of things. In Said El Hajji, Sihem Mesnager, and El Mamoun Souidi, editors, *Codes, Cryptology and Information Security - 4th International Conference, C2SI 2023, Rabat, Morocco, May 29-31, 2023, Proceedings*, volume 13874 of *Lecture Notes in Computer Science*, pages 57–76. Springer, 2023.

[72] Simon Meier, Benedikt Schmidt, Cas Cremers, and David A. Basin. The TAMARIN prover for the symbolic analysis of security protocols. In Natasha Sharygina and Helmut Veith, editors, *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701. Springer, 2013.

[73] Carlos Aguilar Melchor, Nicolas Aragon, Sihem Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Edoardo Persichetti, Gilles Zémor, Joppe Bos, Aurélien Dion, Jérôme Lacan, Jean-Marc Robert, and Pascal Veron. HQC Algorithm Specifications and Supporting Documentation. Technical report, National Institute of Standards and Technology, 2022. 4th Round submission to the NIST's Post-Quantum Cryptography Standardization Process.

[74] National Institute of Standards and Technology. Security (evaluation criteria) — post-quantum cryptography standardization, 2017. Accessed: 2025-06-24.

[75] National Institute of Standards and Technology. Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process. Technical report, National Institute of Standards and Technology, 2022. Available at https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf.

[76] National Institute of Standards and Technology. Sha-3 standard: Permutation-based hash and extendable-output functions. Technical Report Federal Information Processing Standards Publication (FIPS) NIST FIPS 202, U.S. Department of Commerce, Washington, D.C., 2015.

[77] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report Federal Information Processing Standards Publication (FIPS) NIST FIPS 204, U.S. Department of Commerce, Washington, D.C., 2024.

[78] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report Federal Information Process-

ing Standards Publication (FIPS) NIST FIPS 203, U.S. Department of Commerce, Washington, D.C., 2024.

[79] National Institute of Standards and Technology. Stateless hash-based digital signature standard. Technical Report Federal Information Processing Standards Publication (FIPS) NIST FIPS 205, U.S. Department of Commerce, Washington, D.C., 2024.

[80] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. *IACR Cryptol. ePrint Arch.*, page 1447, 2019.

[81] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020, Paris, France, April 15-17, 2020, Proceedings*, volume 12100 of *Lecture Notes in Computer Science*, pages 72–91. Springer, 2020.

[82] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. *IACR Cryptol. ePrint Arch.*, page 1447, 2021.

[83] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May 2022 - 3 June 2022*, pages 727–740. ACM, 2022.

[84] Sebastian Paul, Felix Schick, and Jan Seedorf. Tpm-based post-quantum cryptography: A case study on quantum-resistant and mutually authenticated TLS for iot environments. In Delphine Reinhardt and Tilo Müller, editors, *ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021*, pages 3:1–3:10. ACM, 2021.

[85] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. Generic side-channel attacks on cca-secure lattice-based PKE and KEM schemes. *IACR Cryptol. ePrint Arch.*, page 948, 2019.

[86] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. *RFC*, 8446:1–160, 2018.

[87] Kadir Sabanci and Mumin Cebe. Exploring post-quantum cryptographic schemes for TLS in 5g nb-iot: Feasibility and recommendations. In Paul A. Pavlou, Vishal Midha, Animesh Animesh, Traci A. Carte, Alexandre R. Graeml, and Alanah Mitchell, editors, *29th Americas Conference on Information Systems, AMCIS 2023, Panama City, Panama, August 10-12, 2023*. Association for Information Systems, 2023.

[88] Stefan Santesson and Hannes Tschofenig. Transport layer security (TLS) cached information extension. *RFC*, 7924:1–19, 2016.

[89] Maximilian Schöffel, Frederik Lauer, Carl Christian Rheinländer, and Norbert Wehn. On the energy costs of post-quantum kems in tls-based low-power secure iot. In *IoTDI '21: International Conference on Internet-of-Things Design and Implementation, Virtual Event / Charlottesville, VA, USA, May 18-21, 2021*, pages 158–168. ACM, 2021.

[90] Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1461–1480. ACM, 2020.

[91] Peter Schwabe, Douglas Stebila, and Thom Wiggers. More efficient post-quantum KEMTLS with pre-distributed public keys. In Elisa Bertino, Haya Schulmann, and Michael Waidner, editors, *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4-8, 2021, Proceedings, Part I*, volume 12972 of *Lecture Notes in Computer Science*, pages 3–22. Springer, 2021.

[92] Michael Scott. On TLS for the internet of things, in a post quantum world. *IACR Cryptol. ePrint Arch.*, page 95, 2023.

[93] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*, pages 124–134. IEEE Computer Society, 1994.

[94] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26(5):1484–1509, 1997.

[95] Victor Shoup. OAEP reconsidered. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara,*

California, USA, August 19-23, 2001, Proceedings, volume 2139 of *Lecture Notes in Computer Science*, pages 239–259. Springer, 2001.

[96] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum TLS: an approximate membership querying approach. In Giuseppe Bianchi and Alessandro Mei, editors, *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2022, Roma, Italy, December 6-9, 2022*, pages 35–42. ACM, 2022.

[97] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In Dongsu Han and Anja Feldmann, editors, *CoNEXT '20: The 16th International Conference on emerging Networking EXperiments and Technologies, Barcelona, Spain, December, 2020*, pages 149–156. ACM, 2020.

[98] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020.* The Internet Society, 2020.

[99] Markus Sosnowski, Florian Wiedner, Eric Hauser, Lion Steger, Dimitrios Schoinianakis, Sebastian Gallenmüller, and Georg Carle. The performance of post-quantum TLS 1.3. In Dario Rossi, Stefano Secci, Olivier Bonaventure, and Lili Qiu, editors, *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2023 (Short Papers), Paris, France, December 5-8, 2023*, pages 19–27. ACM, 2023.

[100] Douglas Stebila and Michele Mosca. Post-quantum key exchange for the internet and the open quantum safe project. In Roberto Avanzi and Howard M. Heys, editors, *Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers*, volume 10532 of *Lecture Notes in Computer Science*, pages 14–37. Springer, 2016.

[101] Yutaro Tanaka, Rei Ueno, Keita Xagawa, Akira Ito, Junko Takahashi, and Naofumi Homma. Multiple-valued plaintext-checking side-channel attacks on post-quantum kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(3):473–503, 2023.

[102] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum

TLS 1.3 for resource-constrained embedded devices. *IACR Cryptol. ePrint Arch.*, page 506, 2023.

[103] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices. In Andrea Bartolini, Kristian F. D. Rietveld, Catherine D. Schuman, and Jose Moreira, editors, *Proceedings of the 20th ACM International Conference on Computing Frontiers, CF 2023, Bologna, Italy, May 9-11, 2023*, pages 366–374. ACM, 2023.

[104] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on embedded systems. *IACR Cryptol. ePrint Arch.*, page 1553, 2021.

[105] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. Cryptology ePrint Archive, Paper 2021/1553, 2021.

[106] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. In Chunhua Su, Dimitris Gritzalis, and Vincenzo Piuri, editors, *Information Security Practice and Experience - 17th International Conference, ISPEC 2022, Taipei, Taiwan, November 23-25, 2022, Proceedings*, volume 13620 of *Lecture Notes in Computer Science*, pages 432–451. Springer, 2022.

[107] Rei Ueno, Keita Xagawa, Yutaro Tanaka, Akira Ito, Junko Takahashi, and Naofumi Homma. Curse of re-encryption: A generic power/em analysis on post-quantum kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):296–322, 2022.

[108] Mark N. Wegman and Larry Carter. New hash functions and their use in authentication and set equality. *J. Comput. Syst. Sci.*, 22(3):265–279, 1981.

[109] Bas Westerbaan. The state of the post-quantum internet, Mar 2024.

[110] Jieyu Zheng, Haoliang Zhu, Yifan Dong, Zhenyu Song, Zhenhao Zhang, Yafang Yang, and Yunlei Zhao. Faster post-quantum TLS 1.3 based on ML-KEM: implementation and assessment. In Joaquín García-Alfaro, Rafal Kozik, Michal Choras, and Sokratis K. Katsikas, editors, *Computer Security - ESORICS 2024 - 29th European*

Symposium on Research in Computer Security, Bydgoszcz, Poland, September 16-20, 2024, Proceedings, Part II, volume 14983 of *Lecture Notes in Computer Science*, pages 123–143. Springer, 2024.

[111] Biming Zhou, Haodong Jiang, and Yunlei Zhao. CPA-secure KEMs are also sufficient for post-quantum TLS 1.3. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*, volume 15486 of *Lecture Notes in Computer Science*, pages 433–464. Springer, 2024.

APPENDICES

Appendix A

Example X.509 public-key certificate

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            42:24:4e:c0:18:9f:49:e0:29:0c:3c:af:7d:5e:5d:4e
        Signature Algorithm: ED25519
        Issuer: C=CA, ST=ON, L=Waterloo, CN=*.eng.uwaterloo.ca
        Validity
            Not Before: Jan  1 00:00:00 2025 GMT
            Not After : Jan  1 00:00:00 2035 GMT
        Subject: C=CA, ST=ON, L=Waterloo, CN=*.eng.uwaterloo.ca
        Subject Public Key Info:
            Public Key Algorithm: ED25519
                ED25519 Public-Key:
                pub:
                    b5:c0:a4:e4:5e:85:a0:c7:b4:99:a3:13:b5:91:f0:
                    fd:59:a2:49:3a:3d:a6:8f:d4:3b:37:1d:a5:d6:e5:
                    10:fe
    Signature Algorithm: ED25519
    Signature Value:
        98:89:27:ae:37:ab:6e:22:ab:b1:b3:29:98:f3:31:bf:48:ed:
        c1:45:7b:1a:d8:7d:44:eb:67:ab:17:f7:c6:98:32:6d:5b:83:
        44:66:8f:49:bb:15:22:5d:3a:4e:97:b8:e6:b1:79:9b:4d:a7:
        83:77:95:bc:0f:8b:eb:d1:1d:0d
```

Listing A.1: The content of an X.509 certificate can be inspected using the command `openssl x509 -text -noout -in certificate.pem`.

Appendix B

Cryptography definitions

B.1 Public-key encryption scheme

Syntax A public-key encryption scheme (PKE) is a collection of three routines (KeyGen, Enc, Dec) defined over some plaintext space $\mathcal{M}$ and some ciphertext space $\mathcal{C}$. Key generation $(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda)$ is a randomized routine that returns a keypair consisting of a public encryption key and a secret decryption key. The encryption routine $\texttt{Enc} : (\texttt{pk}, m) \mapsto c$ encrypts the input plaintext m under the input public key pk and produces a ciphertext c. The decryption routine $\texttt{Dec} : (\texttt{sk}, c) \mapsto m$ decrypts the input ciphertext c under the input secret key and produces the corresponding plaintext. Where the encryption routine is randomized, we denote the randomness by a coin $r \in \mathcal{R}$ where $\mathcal{R}$ is called the coin space. Decryption routines are assumed to always be deterministic.

Correctness A PKE is δ-correct if

$$E\left[\max_{m\in\mathcal{M}} P\left[\texttt{Dec}(\texttt{sk}, c) \neq m \mid c \xleftarrow{\$} \texttt{Enc}(\texttt{pk}, m)\right]\right] \leq \delta$$

Where the expectation is taken with respect to the probability distribution of all possible keypairs. For many lattice-based cryptosystems, decryption failures could leak information about the secret key, although the probability of a decryption failure is low enough that classical adversaries cannot exploit decryption failure more than they can defeat the underlying lattice problems.

Security The security of PKE's is conventionally discussed using adversarial games played between a challenger and an adversary [39]. In the OW-ATK game (Figure B.1), the challenger samples a random keypair and a random encryption. The adversary is given the public key, the random encryption (also called the challenge ciphertext), and access to ATK, then asked to decrypt the challenge ciphertext.

OW-ATK game

$(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda)$
$m^* \xleftarrow{\$} \mathcal{M}$
$c^* \xleftarrow{\$} \texttt{Enc}(\texttt{pk}, m)$
$\hat{m} \xleftarrow{\$} A^{\text{ATK}}(1^\lambda, \texttt{pk}, c^*)$
return $[\![\hat{m} = m^*]\!]$

Figure B.1: The one-wayness game: challenger samples a random keypair and a random encryption, and the adversary wins if it correctly produces the decryption

The advantage of an adversary is its probability of producing the correct decryption: $\text{Adv}_{\texttt{PKE}}^{\text{OW-ATK}}(A) = P[\hat{m} = m^*]$. A PKE is said to be OW-ATK secure if no efficient adversary can win the OW-ATK game with non-negligible probability.

IND-ATK game

$(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda)$
$(m_0, m_1) \xleftarrow{\$} A^{\text{ATK}}(1^\lambda, \texttt{pk})$
$b \xleftarrow{\$} \{0, 1\}$
$c^* \xleftarrow{\$} \texttt{Enc}(\texttt{pk}, m_b)$
$\hat{b} \xleftarrow{\$} A^{\text{ATK}}(1^\lambda, \texttt{pk}, c^*)$
return $[\![\hat{b} = b]\!]$

Figure B.2: IND-ATK game: adversary is asked to distinguish the encryption of one message from another

In the IND-ATK game (Figure B.2), the adversary chooses two distinct messages and receives the encryption of one of them, randomly selected by the challenger. The advantage of an adversary is its probability of correctly distinguishing the ciphertext of one message from the other beyond blind guess: $\text{Adv}_{\texttt{PKE}}^{\text{IND-ATK}}(A) = |P[\hat{b} = b] - \frac{1}{2}|$. A PKE is said to be IND-ATK secure if no efficient adversary can win the IND-ATK game with non-negligible advantage.

$\mathcal{O}^{\texttt{Dec}}(c)$
1: **return** $\texttt{Dec}(\texttt{sk}, c)$

$\mathcal{O}^{\texttt{PCO}}(m, c)$
1: **return** $[\![\texttt{Dec}(\texttt{sk}, c) = m]\!]$

Figure B.3: Decryption oracle $\mathcal{O}^{\texttt{Dec}}$ (left) answers decryption queries by returning the decryption of the queried ciphertext. Plaintext-checking oracle $\mathcal{O}^{\texttt{PCO}}$ (right) answers whether the queried plaintext is the decryption of the queried ciphertext.

In public-key cryptography, all adversaries are assumed to have access to the public key (ATK = CPA). If the adversary has access to a decryption oracle, it is said to mount chosen-ciphertext attack (ATK = CCA). If the adversary has access to a plaintext-checking oracle (PCO) , then it is said to mount plaintext-checking attack (ATK = PCA). See Figure B.3 for algorithmic description of the oracles.

$$\text{ATK} = \begin{cases} \text{CPA} & \mathcal{O}^{\text{ATK}} = \cdot \\ \text{PCA} & \mathcal{O}^{\text{ATK}} = \mathcal{O}^{\texttt{PCO}} \\ \text{CCA} & \mathcal{O}^{\text{ATK}} = \mathcal{O}^{\texttt{Dec}} \end{cases}$$

B.2 Key encapsulation mechanism

Syntax A key encapsulation mechanism (KEM) is a collection of three routines $(\texttt{KeyGen}, \texttt{Encap}, \texttt{Decap})$ defined over some ciphertext space $\mathcal{C}$ and some key space $\mathcal{K}$. Key generation $\texttt{KeyGen} : 1^\lambda \mapsto (\texttt{pk}, \texttt{sk})$ is a randomized routine that returns a keypair. Encapsulation $\texttt{Encap} : \texttt{pk} \mapsto (c, K)$ is a randomized routine that takes a public encapsulation key and returns a pair of ciphertext c and shared secret K (also commonly referred to as session key). Decapsulation $\texttt{Decap} : (\texttt{sk}, c) \mapsto K$ is a deterministic routine that uses the secret key $\texttt{sk}$ to recover the shared secret K from the input ciphertext c. Where the KEM chooses to reject invalid ciphertext explicitly, the decapsulation routine can also output the rejection symbol $\perp$. We assume a KEM to be perfectly correct:

$$P\left[\texttt{Decap}(\texttt{sk}, c) = K \mid (\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda); (c, K) \xleftarrow{\$} \texttt{Encap}(\texttt{pk})\right] = 1$$

Security Similar to PKE security, the security of KEM is discussed using adversarial games. In the IND-ATK game (Figure B.4), the challenger generates a random keypair and encapsulates a random secret; the adversary is given the public key and the ciphertext, then asked to distinguish the shared secret from a random bit string.

KEM IND-ATK Game

$(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}((1^\lambda))$
$(c^*, K_0) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk})$
$K_1 \xleftarrow{\$} \mathcal{K}$
$b \xleftarrow{\$} \{0, 1\}$
$\hat{b} \xleftarrow{\$} A^{\mathtt{ATK}}(1^\lambda, \mathtt{pk}, c^*, K_b)$
return $[\![\hat{b} = b]\!]$

Figure B.4: The IND-ATK game for KEM

The advantage of an adversary is its probability of winning beyong blind guess. A KEM is said to be IND-ATK secure if no efficient adversary can win the IND-ATK game with non-negligible advantage.

$$\mathtt{Adv}^{\mathtt{IND\text{-}ATK}}(A) = \left| P\left[A^{\mathtt{ATK}}(1^\lambda, c^*, K_b) = b \;\middle|\; \begin{array}{r} (\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}(1^\lambda); \\ (c^*, K_0) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk}); \\ K_1 \xleftarrow{\$} \mathcal{K}; b \xleftarrow{\$} \{0, 1\} \end{array} \right] - \frac{1}{2} \right|$$

By default, all adversaries are assumed to have the public key, with which they can mount chosen plaintext attacks (ATK = CPA). If the adversary as access to a decapsulation oracle $\mathcal{O}^{\mathtt{Decap}} : c \mapsto \mathtt{Decap}(\mathtt{sk}, c)$, it is said to mount a chosen-ciphertext attack (ATK = CCA).

B.3 Message authentication code

Syntax A message authentication code (MAC) is a collection of two routines `Sign` and `Verify` defined over some key space $\mathcal{K}$, some message space $\mathcal{M}$, and some tag space $\mathcal{T}$. The signing routine $\mathtt{Sign} : (k, m) \mapsto t$ authenticates the message m under the symmetric key k by producing a tag t. The verification routine $\texttt{Verify}(k, m, t)$ outputs 1 if the message-tag pair (m, t) is authentic under the symmetric key k and 0 otherwise. Many MAC constructions are deterministic: for these constructions it is simpler to denote the signing routine by $t \leftarrow \texttt{MAC}(k, m)$, and verification done using a simple comparison. Some MAC constructions require a distinct or randomized nonce $r \xleftarrow{\$} \mathcal{R}$, and the signing routine will take this additional argument $t \leftarrow \texttt{MAC}(k, m; r)$.

Security The standard security notion for a MAC is *existential unforgeability under chosen message attack (EUF-CMA)*. We define it using an adversarial game in which an adversary has access to a signing oracle $\mathcal{O}^{\texttt{Sign}} : m \mapsto \texttt{Sign}(k, m)$ and tries to produce a valid message-tag pair that has not been queried from the signing oracle (Figure B.5).

MAC EUF-CMA game
1: $k^* \xleftarrow{\$} \mathcal{K}$
2: $(\hat{m}, \hat{t}) \xleftarrow{\$} A^{\texttt{CMA}}()$
3: **return** $[\![\texttt{Verify}(k^*, \hat{m}, \hat{t}) \wedge (\hat{m}, \hat{t}) \notin \mathcal{O}^{\texttt{Sign}}]\!]$

Figure B.5: The signing oracle signs the queried message with the secret key. The adversary must produce a message-tag pair that has never been queried before

The advantage of the adversary is the probability that it successfully produces a valid message-tag pair. A MAC is said to be EUF-CMA secure if no efficient adversary has non-negligible advantage. Some MACs are *one-time existentially unforgeable* (we call them one-time MAC), meaning that each secret key can be used to authenticate exactly one message. The corresponding security game is identical to the EUF-CMA game except for that the signing oracle will only answer up to one query.

Appendix C

ML-KEM

Below are high-level summaries of the PKE sub-routines and the KEM routines of ML-KEM:

`K-PKE.KeyGen()`

1: $A \xleftarrow{\$} R_q^{k \times k}$
2: $\mathbf{s} \xleftarrow{\$} \mathcal{X}_{\eta_1}^k$
3: $\mathbf{e} \xleftarrow{\$} \mathcal{X}_{\eta_1}^k$
4: $\mathbf{t} \leftarrow A\mathbf{s} + \mathbf{e}$
5: $\mathtt{pk} \leftarrow (A, \mathbf{t})$
6: $\mathtt{sk} \leftarrow \mathbf{s}$
7: **return** $(\mathtt{pk}, \mathtt{sk})$

`K-PKE.Enc(pk, m)`

Ensure: $\mathtt{pk} = (A, \mathbf{t})$
Ensure: $m \in R_2$
1: $\mathbf{r} \xleftarrow{\$} \mathcal{X}_{\eta_1}^k$
2: $\mathbf{e}_1 \xleftarrow{\$} \mathcal{X}_{\eta_2}^k$
3: $e_2 \xleftarrow{\$} \mathcal{X}_{\eta_2}$
4: $\mathbf{c}_1 \leftarrow A\mathbf{r} + \mathbf{e}_1$
5: $c_2 \leftarrow \mathbf{t}^\intercal \mathbf{r} + e_2 + m \cdot \lfloor \frac{q}{2} \rceil$
6: **return** $(\mathbf{c}_1, c_2)$

`K-PKE.Dec(sk, c)`

Ensure: $c = (\mathbf{c}_1, c_2)$
Ensure: $\mathtt{sk} = \mathbf{s}$
1: $\hat{m} \leftarrow c_2 - \mathbf{c}_1^\intercal \cdot \mathbf{s}$
2: $\hat{m} \leftarrow \mathtt{Round}(\hat{m})$
3: **return** $\hat{m}$

Figure C.1: K-PKE is an IND-CPA secure PKE based on the conjectured intractability of the Module Learning with Error (MWLE) problem

$\texttt{ML-KEM.KeyGen()}$

$(\texttt{pk}, \texttt{sk}') \xleftarrow{\$} \texttt{K-PKE.KeyGen()}$
$h \leftarrow \texttt{SHA3-256}(\texttt{pk})$
$z \xleftarrow{\$} \mathcal{M}$
$\texttt{sk} \leftarrow (\texttt{sk}', \texttt{pk}, h, z)$
return $(\texttt{pk}, \texttt{sk})$

$\texttt{ML-KEM.Encap}(\texttt{pk})$

$m \xleftarrow{\$} \mathcal{M}$
$h \leftarrow \texttt{SHA3-256}(\texttt{pk})$
$(K, r) \leftarrow \texttt{SHA3-512}(m, h)$
$c \leftarrow \texttt{K-PKE.Enc}(\texttt{pk}, m, r)$
return (c, K)

$\texttt{ML-KEM.Decap}(\texttt{sk}, c)$

$\hat{m} \leftarrow \texttt{K-PKE.Dec}(\texttt{sk}', c)$
$(\overline{K}, \hat{r}) \leftarrow \texttt{SHA3-512}(\hat{m}, h)$
$\hat{c} \leftarrow \texttt{K-PKE.Enc}(\texttt{pk}, m, \hat{r})$
if $\hat{c} = c$ **then**
    $K \leftarrow \overline{K}$
else
    $K \leftarrow \texttt{SHA3-256}(c, z)$
end if
return K

Figure C.2: ML-KEM uses the $\texttt{KEM}_m^{\not\bot}$ variant of the modular Fujisaki-Okamoto KEM transformation